Diffie–Hellman Key Exchange from Commutativity to Group Laws

Dung Hoang Duong* Youming Qiao† Chuanqi Zhang‡

Abstract

In Diffie–Hellman key exchange, the commutativity of power operations is instrumental in the agreement of keys. Viewing commutativity as a law in abelian groups, we propose Diffie–Hellman key exchange in the group action framework (Brassard–Yung, *Crypto*'90; Ji–Qiao–Song–Yun, *TCC*'19), for actions of non-abelian *groups with laws*. The security of this protocol is shown, following Fischlin, Günther, Schmidt, and Warinschi (*IEEE S&P*'16), based on a pseudorandom group action assumption. A concrete instantiation is proposed based on the monomial code equivalence problem.

1 Introduction

1.1 Background

Diffie–Hellman key exchange and commutativity. The celebrated Diffie–Hellman key exchange [DH76] is a public-key protocol allowing two parties to agree on a common key. This protocol is of significant historical importance and is widely used in practice.

Let us quickly review this protocol. For a prime p, let C_p be a cyclic group of order p and $\gamma \in C_p$ a generator of C_p. Alice (resp. Bob) randomly sample $a \in [p-1] := \{1, \ldots, p-1\}$ (resp. $b \in [p-1]$). Alice computes γ^a and sends it to Bob. Bob computes γ^b and sends it to Alice. Alice computes $(\gamma^b)^a = \gamma^{ba}$ as her key. Bob computes $(\gamma^a)^b = \gamma^{ab}$ as his key.

It is readily seen that the *commutativity* of $ab = ba$ ensures that $\gamma^{ba} = \gamma^{ab}$ so Alice and Bob share the same key. The role of commutativity is natural and crucial, so various attempts to generalize Diffie–Hellman tried to exploit commutativity in one way or another, even when dealing with non-commutative objects [MSU11, PN17].

An intriguing question is then whether commutativity could be generalized or relaxed for reaching key agreement in the Diffie–Hellman-type key exchange protocols. Our goal in this article is to propose a natural generalization via *group laws* in the framework of *group action based cryptography* [BY90, JQSY19].

Group action based cryptography. Cryptography based on group actions was first studied as a framework by Brassard and Yung [BY90], who proposed the definition of one-way group actions and presented some cryptographic applications. Couveignes developed this framework further but

*Institute of Cybersecurity and Cryptology, School of Computing and Information Technology, University of Wollongong, Australia (hduong@uow.edu.au). Research supported in part by Australian Research Council LP220100332.

†Centre for Quantum Software and Information, University of Technology Sydney, Australia (Youming.Qiao@uts.edu.au). Research supported in part by Australian Research Council DP200100950 and LP220100332.

‡Centre for Quantum Software and Information, University of Technology Sydney, Australia (Chuanqi.Zhang@uts.edu.au). Research supported by Australian Research Council DP200100950 and LP220100332, and the Sydney Quantum Academy, Sydney, NSW, Australia.

with a focus on commutative groups in [Cou06], and one contribution of Couveignes is the proposal of using class group actions on elliptic curves in cryptography.

Diffie–Hellman key exchange can be formulated via group actions, as observed by Brassard and Yung [BY90] and Couveignes [Cou06]. Using the notation in the description above, define the set S to be $C_p \setminus \{\mathrm{id}\}$, the cyclic group C_p excluding the identity element id. The group $G := \mathrm{Aut}(C_p)$ is the automorphism group of C_p. Note that G is isomorphic to $(\mathbb{Z}/p\mathbb{Z})^*$, the multiplicative group of $\mathbb{Z}/p\mathbb{Z}$, so we can identify $a \in G$ as an element in $(\mathbb{Z}/p\mathbb{Z})^*$. The group action is then $a \in (\mathbb{Z}/p\mathbb{Z})^*$ sending $\delta \in S = C_p \setminus \{\mathrm{id}\}$ to δ^a.

Recently, two works [JQSY19, AFMP20] further developed this framework by introducing new notions such as pseudorandom group actions (which naturally generalizes the Decisional Diffie–Hellman assumption [Bon98]) and devising more cryptographic applications. In addition, more candidate group actions suitable for cryptographic uses were proposed. In [JQSY19], the general linear group action on tensors was suggested as a candidate for pseudorandom group actions. In [AFMP20], the group actions are class group actions with variations such as in [CLM+18, BKV19].

A major application of group action based cryptography is digital signatures. This construction is based on the Goldreich–Micali–Wigderson (GMW) zero-knowledge protocol for graph isomorphism [GMW91] and the Fiat–Shamir (FS) transformation [FS86]. This GMW-FS construction has seen a recent revival as evidenced by the works [BMPS20, CNP+23, TDJ+22] for actions by symmetric and general linear groups, and the works [BKV19, FG19] for class group actions. See also [BGZ23, BCD+24a, BDFGP23] for recent progress and surveys.

Concrete group actions used in cryptography. We recall some concrete group actions proposed for use in cryptography, besides the group action underlying discrete logarithm.

On the commutative (abelian) group side, a main candidate is the action of the ideal-class group $\mathrm{cl}(\mathcal{O})$ on the set of $\mathbb{F}_q$-isomorphism classes of ordinary elliptic curves over $\mathbb{F}_q$ whose endomorphism ring is a given order $\mathcal{O}$ in an imaginary quadratic field. It was later adapted by Castryck et al. [CLM+18] to the case of supersingular elliptic curves resulting in an efficient key exchange protocol called CSIDH. Based on CSIDH, many isogeny-based cryptographic constructions are built, such as digital signatures SeaSign [FG19] and CSI-FiSh [BKV19], threshold signature [FM20], ring signatures [BKP20], group signatures [BDK+22], blind signatures [KLLQ23], updatable encryption [LR24], and password-authenticated key exchange (PAKE) [AEK+22, IY23].

On the non-commutative (non-abelian) group side, there are three problem families, namely linear code equivalence [Beu20, BMPS20], tensor isomorphism and its variations [JQSY19, GQ19, CNP+23, TDJ+22], and lattice isomorphism [HR14, BBCDK24, DPPvW22]. These are based on actions by monomial groups, general linear groups over finite fields, and general linear groups over $\mathbb{Z}$. Four digital signature schemes, LESS [BMPS20], MEDS [CNP+23], ALTEQ [BDN+23], and Hawk [DPPvW22], were submitted to the NIST's call for post-quantum digital signature schemes, with LESS and Hawk making into round 2.

Comparisons of some cryptographic commutative and non-commutative group actions. When groups are commutative, then key exchange protocols and public-key encryptions can be realized following Diffie–Hellman [DH76] and ElGamal [ElG85].

When groups are non-commutative, key exchange and public-key encryption seem much harder to achieve. To devise a public-key encryption scheme based on non-commutative group actions was proposed in [JQSY19] as an open problem. In a recent breakthrough [HMY23], Hhan, Morimae and Yamakawa proposed the first public-key encryption scheme based on cryptographic non-commutative group actions, albeit requiring the ciphertexts to be quantum. Regarding key ex-

change, to the best of our knowledge, there were no proposals in vein of the Diffie–Hellman key exchange protocols before this work. Indeed, as discussed above, commutativity is used crucially in the original Diffie–Hellman protocol to ensure the key agreement.

While commutative groups admit more cryptographic functionalities more easily, there are several reasons to pursue non-commutative group actions, in particular in post-quantum cryptography.

In the context of quantum algorithms, algorithmic problems underlying commutative group actions can usually be formulated as instances of the abelian hidden shift problem [CvD10]. This allows for adapting the Kuperberg's dihedral hidden subgroup algorithm [Kup05] to obtain quantum subexponential-time algorithms such as for constructing elliptic curve isogenies [CJS14] and some recent attacks [Pei20, BS20] on CSIDH [CLM+18].

On the other hand, the non-abelian group actions used in cryptography, such as linear code equivalence [BMPS20] and tensor isomorphism [JQSY19, CNP+23, TDJ+22], can be cast as instances of the hidden subgroup problem with the ambient groups being symmetric and general linear groups. For such hidden subgroup problems, there is strong negative evidence [HMR+10, MRS10, DMR11, DMR15] for using the standard techniques from Shor's algorithms [Sho94] and Kuperberg's sieve techniques [Kup05] to obtain a polynomial-time or even subexponential-time quantum algorithm. These are referred to as the "strongest such insights we have about the limits of quantum algorithms" by Moore, Russell, and Vazirani [MRV07].

Besides these considerations from quantum algorithms, the group action computation for CSIDH is relatively slow, and to speed it up requires considerable research (see e.g. [BKV19, Hou25]). On the other hand, those non-abelian group actions based on lattices, tensors, and linear codes are fast to compute.

Quantum cryptography based on group actions. More advanced functionalities based on non-abelian group actions can be achieved in the context of quantum cryptography. We already mentioned the public-key encryption scheme with quantum ciphertexts by Hhan, Morimae and Yamakawa [HMY23]. Another exciting recent development is the quantum money scheme based on non-abelian group action by Bostanci, Nehoran, and Zhandry [BNZ25].

1.2 Key exchange from commutativity to group laws: the framework

Our goal in this paper is to propose one approach to extend Diffie–Hellman key exchange protocols to beyond commutativity. The key notion here is the so-called group laws.

Group laws. A *law in a group* G is an equation that is satisfied by any assignments of variables by group elements in G. For example, $xy = yx$ is a law in abelian groups, or put in another way, abelian groups are groups satisfying the commutative law $xy = yx$. Therefore, on the one hand, laws can be used to define group classes, such as abelian groups, metabelian groups, and solvable groups. On the other hand, there was considerable recent research on short laws for almost simple groups (such as symmetric groups) and general finite groups [KT16, Tho17, BT19], and these works serve as a guidance for several considerations in this paper.

With the commutative law in mind, we reformulate Diffie–Hellman key exchange for abelian group actions as follows. Suppose an abelian group G acts on a set S from the right. For $g \in G$ and $s \in S$, we use $s * g$ to denote the result of $g \in G$ acting on $s \in S$. Following Diffie–Hellman, Alice randomly samples $a \in G$ and Bob samples $b \in G$. Alice then computes $s * a$ and sends it to Bob, while Bob computes $s * b$ and sends it to Alice. Alice and Bob computes $(s * b) * a$ and $(s * a) * b$ respectively and they reach a common key $(s * b) * a = s * (ba) = s * (ab) = (s * a) * b$, by group action axioms and the commutativity law. This formulation was observed independently

by Couveignes [Cou06] and Rostovtsev–Stolbunov [RS06, Sto10] in the context of isogeny-based cryptography.

Key exchange for general group actions. The Diffie–Hellman key exchange protocol can be readily generalized to actions of groups with laws. Indeed, let G be a group satisfying a law, that is an equation, of the form $u(a, b, c, \dots) = v(a, b, c, \dots)$, where u and v are words in variables $a, b, c, \dots$ and their inverses. Then by assigning these variables to Alice and Bob appropriately, they can sample group elements from G, communicate according to u to get Alice's key, and communicate according to v to get Bob's key. The agreement of their keys is then ensured by the group law.

In Section 3, we formally present the above key exchange protocol for actions of groups with laws, and prove its security [BR93, FGSW16], based on a *pseudorandom group action* assumption. One simplification we make there is to focus on laws involving just two variables, as a law in k variables can be transformed into another law in 2 variables of length polynomially bounded by the original law [BT19].

A natural question is then whether general groups admit some laws. That is, whether certain groups are "outlaws." To start with, note that finite groups always admit some laws. For example, for a finite group G of order N, a naive law is $x^N = \mathrm{id}$. Shorter laws with two variables exist, as Bradford and Thom showed the existence of laws in two variables of length $\tilde{O}(N^{2/3})$ for groups of order N [BT19]. Still, these laws would not be useful for our purpose, as the group order N is not polynomially bounded.

Key exchange for metabelian group actions. We now present the key exchange protocol for actions by metabelian groups. We believe that it is a nice example to illustrate the above ideas concretely. However, to the best of our knowledge, we do not have candidate cryptographic actions by metabelian groups; therefore, we will move to an instantiation based on symmetric group actions in the next subsection. We leave the search for cryptographic metabelian group actions as an intriguing open problem.

Recall that a group G is *metabelian*, if for any $a, b, c, d \in G$,

$$aba^{-1}b^{-1}cdc^{-1}d^{-1} = cdc^{-1}d^{-1}aba^{-1}b^{-1}. \tag{1}$$

That is, the *metabelian* law is $xyx^{-1}y^{-1}uvu^{-1}v^{-1} = uvu^{-1}v^{-1}xyx^{-1}y^{-1}$ in variables x, y, u, v.

Denoting by $[a, b]$ the commutator of a, b, Equation (1) is equivalent to $[a, b][c, d] = [c, d][a, b]$, which is further equivalent to $[[a, b], [c, d]] = \mathrm{id}$. That is, metabelian groups are solvable groups of derived length ≤ 2.[1] For example, dihedral groups and affine maps $x \to ax + b$ over a field are metabelian.

Suppose a metabelian group G acts on a set S, with $g \in G$ sending $s \in S$ to $s * g \in S$. While Equation (1) can be used for Alice and Bob to reach a key agreement (see below), we need to be cautious about assigning a, b, c, d to Alice and Bob appropriately. Indeed, some assignments of $a, b, c, d \in G$ to Alice and Bob would get us back to the commutative setting, such as a and b to Alice, and c and d to Bob. Furthermore, we need to make sure that one sequence of communications ends with Alice holding the last group element, and the other sequence of communications ends with Bob holding the last group element, so they can apply these last elements to get their share of the key.

With these considerations in mind, we arrive at the following protocol.

[1] Briefly speaking, the derived length of a group refers to the number of steps needed to reach the trivial group {id} when repeatedly taking commutator subgroups.

1. Fix $s \in S$ as the public key.
2. Alice randomly samples $a, d \in G$, and Bob randomly samples $b, c \in G$, as their private keys.
3. Alice initiates $s * a$, and communicates with Bob in the following three rounds:
   (a) Alice $\xrightarrow{s*a}$ Bob $\xrightarrow{(s*a)*b}$ Alice. Let $t_1 = s * (ab)$.
   (b) Alice $\xrightarrow{t_1*a^{-1}}$ Bob $\xrightarrow{(t_1*a^{-1})*(b^{-1}c)}$ Alice. Let $t_2 = t_1 * (a^{-1}b^{-1}c)$.
   (c) Alice $\xrightarrow{t_2*d}$ Bob $\xrightarrow{(t_2*d)*c^{-1}}$ Alice. Let $t_3 = t_2 * (dc^{-1})$.

   Alice then computes $t_4 = t_3 * d^{-1}$ as her secret key.
4. Bob initiates $s * c$, and communicates with Alice in the following three rounds.
   (a) Bob $\xrightarrow{s*c}$ Alice $\xrightarrow{(s*c)*d}$ Bob. Let $r_1 = s * (cd)$.
   (b) Bob $\xrightarrow{r_1*c^{-1}}$ Alice $\xrightarrow{(r_1*c^{-1})*d^{-1}a}$ Bob. Let $r_2 = r_1 * (c^{-1}d^{-1}a)$.
   (c) Bob $\xrightarrow{r_2*b}$ Alice $\xrightarrow{(r_2*b)*a^{-1}}$ Bob. Let $r_3 = r_2 * (ba^{-1})$.

   Bob then computes $r_4 = r_3 * b^{-1}$ as his secret key.

Note that by assigning a, d to Alice and b, c to Bob, elements of the form $s * ([a, b])$ or $s * ([c, d])$ are never revealed during the execution of the protocol. This ensures that we do not get back to the commutative setting.

It is clear that Alice and Bob reach a key agreement: in Step 3, Alice and Bob follow the sequence on the left-hand side of Equation (1), while in Step 4, Alice and Bob follow the sequence on the right-hand side of Equation (1). Therefore, t_4 and r_4, that are the secret keys of Alice and Bob, are the same.

The downside is that Alice and Bob need 6 rounds of communications. Alice performs 7 group actions, and so does Bob. In general, the group action is usually assumed to be efficiently computable. Recall that in the original Diffie–Hellman protocol, Alice and Bob only need 1 round of communication, and each of them performs 2 group actions. Still, when our interest is more on exploring the theoretical possibility of making use of non-commutative group actions, we could even accommodate polynomially many rounds of communications.

A more serious problem is that we don't know of concrete cryptographic metabelian group actions. That is, a group action suitable for cryptographic uses needs to demonstrate evidence for satisfying one-way [BY90] or pseudorandom [JQSY19] assumptions. At present, candidate cryptographic group actions are either abelian from isogeny based cryptography [CLM+18, BKV19], or highly non-abelian (almost simple) groups such as symmetric or general linear groups [JQSY19, TDJ+22, CNP+23, BMPS20]. Indeed, it would be of great interest to identify candidate cryptographic actions by non-abelian solvable groups.

1.3 Key exchange from commutativity to group laws: a concrete proposal

Key exchange for symmetric group actions. When we turn to symmetric groups S_n, the group of permutations of $[n] = \{1, 2, \ldots, n\}$, some recent nice progress in group theory comes to our aid. In [KT16], Kozma and Thom demonstrated laws for S_n in two variables of length $\exp(\tilde{O}(\log(n)^4))$. This result builds on several important works, including [Lie84, HS14]. Actually,

if a well-known conjecture of Babai [BS92] holds, the law length can be brought further down to $n^{O(\log\log(n))}$.

While the laws in [KT16] are close to polynomial in length, they may still be a bit too complicated to use in cryptographic protocols. Looking into the proofs in [KT16], an elementary fact about S_n turns out to be crucial: $1/n$ fraction of permutations are full-cycles. This means that $(xy)^n = \mathrm{id}$, or equivalently $(xy)^{\lceil n/2\rceil} = (y^{-1}x^{-1})^{\lfloor n/2\rfloor}$, is a law that holds with probability (at least) $1/n$ when x and y are randomly sampled from S_n.[2]

Such probabilistic laws have been studied in the literature under the name of "almost laws", and almost laws in two variables for S_n of length $\tilde{O}(n^8)$ were presented in [Zyr20] (see [BT19]). Here, we shall keep using $(xy)^{\lceil n/2\rceil} = (y^{-1}x^{-1})^{\lfloor n/2\rfloor}$ due to its simplicity and efficiency (length n compared to length $\tilde{O}(n^8)$ as in [Zyr20]).

This leads to the following protocol for key exchange based on symmetric group actions. Let S_n act on a set S, with $g \in \mathrm{S}_n$ sending $s \in S$ to $s * g$. Alice randomly samples $g \in \mathrm{S}_n$ and Bob randomly samples $h \in \mathrm{S}_n$. They agree on $s \in S$ which can be made public. For Bob's key, Alice and Bob compute and communicate $s * g$, $s * gh$, $\ldots$, $s * (gh)^{\lceil n/2\rceil - 1}$ in turn. In the last round, Alice sends $s * ((gh)^{\lceil n/2\rceil - 1}g)$ to Bob, and Bob computes $s * ((gh)^{\lceil n/2\rceil})$ as his key. For Alice's key, Bob and Alice compute and communicate $s * h^{-1}$, $s * (h^{-1}g^{-1})$, $\ldots$, $s * (h^{-1}g^{-1})^{\lfloor n/2\rfloor - 1}$ in turn. In the last round, Bob sends $s * (h^{-1}g^{-1})^{\lfloor n/2\rfloor - 1}h^{-1}$ to Alice, and Alice computes $s * (h^{-1}g^{-1})^{\lfloor n/2\rfloor}$ as her key.

In the case of $(gh)^n = \mathrm{id}$, such as gh being a full-cycle permutation, $s * ((gh)^{\lceil n/2\rceil}) = s * (h^{-1}g^{-1})^{\lfloor n/2\rfloor}$, that is, the keys of Alice and Bob agree. While the probability for it to hold is $1/n$, it suffices for a theoretical demonstration. We then apply a Blackbox transformation from [FGSW16] to obtain key confirmation between Alice and Bob, and if it does not satisfy the key confirmation check, we will repeat the key exchange protocol by choosing a new pair of g and h, until $(gh)^n = \mathrm{id}$ (cf. Section 5).

Review of the group action underlying linear code equivalence. To instantiate the above key exchange protocol, we need a concrete symmetric group action suitable for cryptographic uses.

Symmetric group actions are an important topic in combinatorics and theoretical computer science. For example, the celebrated graph isomorphism problem can be formulated as the orbit problem of S_n acting on sets of size-2 subsets of $[n]$ [Luk82]. Unfortunately (for cryptographers), graph isomorphism turns out to be (almost) easy both in practice [MP14] and in theory [Bab16].

On the other hand, the *linear code equivalence* problem is generally regarded as a candidate for cryptographic group actions. This problem asks whether two linear spaces are the same up to permutation of, and scalar multiplications on, the coordinates. In the literature, this version of the problem is usually referred to as *monomial code equivalence*, to distinguish from *permutation code equivalence*, in which scalar multiplications are not considered.

Several works show that permutation code equivalence is easy in some cases [Sen00, BOS19]. Therefore, monomial code equivalence is preferred for cryptographic uses, as the current best practical algorithms for monomial code equivalence are the ones exploiting small-weight vectors by Leon and Beullens [Leo82, Beu20]. For algorithms with worst-case asymptotic analyses, some recent works on monomial code equivalence design faster but still exponential-time algorithms [BBB+25], by exploiting some ideas from Babai's algorithm on permutation code equivalence [BCGQ11]. Based on monomial code equivalence, the digital signature scheme LESS [BMPS20] was developed and submitted to the call for additional post-quantum digital signatures of NIST [BBB+23].

[2] Note that when x and y are assigned with uniformly sampled random permutations, their product xy is also a uniformly random permutation.

Issues of using monomial code equivalence. Directly using monomial code equivalence has two issues.

First, it is a monomial group action instead of a symmetric group action. This could be fixed relatively easily by utilising group laws for the monomial group.

The second and more serious issue is the following. During our protocol, a secret key needs to be used several times. Recall that it is straightforward to cast monomial code equivalence as the monomial group $\mathrm{Mon}(n, q)$ acting on the set of m-dimensional codes in $\mathbb{F}_q^n$. With this modelling, in cryptographic protocols, the secret key is the monomial matrix M, which is a product of a diagonal matrix D and a permutation matrix P (as in e.g. [BMPS20]). A recent surprising discovery is that the secret monomial matrix cannot be used several times, not even twice [DDS24, BCD+24b, BN25]. The secret key reusing is the main bottleneck for using monomial code equivalence in our key exchange protocols.

The remedy: viewing monomial code equivalence as a symmetric group action. In [DKQ+25], it was shown that a symmetric group action can be extracted from the monomial code equivalence problem. Furthermore, it was suggested there that the techniques in [DDS24, BCD+24b, BN25] for breaking multiple uses of the secret keys in the monomial group action setting do not carry over to this symmetric group setting, at least not directly; see Section 4.2 for a slightly expanded discussion on this issue. This makes that group action suitable for instantiating the key exchange protocol as above. In this paper, we carry out further cryptanalysis experiments to support its security in this multiple key use setting in Section 4.3.

For readers' convenience, we briefly review how to extract the symmetric group action from monomial code equivalence from [BCD+24b, BN25] here, with further details in Section 4.1. Let $\mathrm{Mat}(d \times n, q)$ be the linear space of $d \times n$ matrices over $\mathbb{F}_q$, the finite field of order q. Let $A, B \in \mathrm{Mat}(d \times n, q)$, with $d \in [n]$. We view A and B as generator matrices of d-dimensional codes in $\mathbb{F}_q^n$. We say that A and B are monomially equivalent as codes, if there exists an $d \times d$ invertible matrix L, an $n \times n$ diagonal matrix D, and an $n \times n$ permutation matrix P, such that $LADP = B$.

To interpret this as a symmetric group action of S_n (i.e., as a permutation matrix P as above), the underlying set S is the set of equivalence classes of $\mathrm{Mat}(d \times n, q)$ under the action of left-multiplying invertible matrices (i.e., L as above) and right-multiplying diagonal matrices (i.e., D as above). So we first need to ensure that this group action is well-defined (Proposition 3). At the end of the protocol, Alice and Bob obtain possibly different matrices S and T, with the guarantee that S and T are in the same equivalence class. As a result, they need to run a canonical form algorithm to arrive at the same representative in this equivalence class (Proposition 4).

1.4 Remarks on some technical aspects

Security models for key exchange. There has been a long history in designing and analyzing the security of key exchange protocols. There are basically three approaches for defining the security of key exchange protocols [DDMW06]: indistinguishability approach [BR93], simulation-based security paradigm [Sho99, BCK98], and universal composability [Can01] or reactive simulatability [PW01].

In this paper, we follow Fischlin et al.'s approach [FGSW16] which follows the previous work by Bellare and Rogaway [BR93] via the indistinguishability approach. The reason is that in our protocol, especially in the instantiation, we need to make sure that the shared keys obtained from two parties after the execution of the session are the same, and hence we need our key exchange protocol to have *key confirmation* property. Basically, following [FGSW16], one needs a key exchange protocol Π that provides *key secrecy* and *match security* properties (see Section 2.4 for details),

then Fischlin et al. [FGSW16] provide a generic transformation to obtain a key exchange preserving those two properties with additional key confirmation property. We call such a transformation applied to Π a Blackbox which we will use abstractly in our instantiation.

New security assumptions. Let us briefly recall the idea of the security proof for the Diffie–Hellman protocol from which our protocol follows. Let C_p be a cyclic group of order p and $\gamma \in C_p$ a generator of C_p. Recall from Section 1.1 that, the *transcript* of the protocol that the adversary can obtain consists of γ^a sent from Alice and γ^b sent from Bob, in addition to the public parameter γ. The key generated by the protocol is $K = \gamma^{ab}$ which can be computed by both Alice and Bob after the interaction.

To obtain the shared key K, the adversary can either solve the Discrete Logarithm Problem (DLP) (i.e., obtain a given γ^a and then compute $K = (\gamma^b)^a$), or solve the Computational Diffie–Hellman (CDH) Problem (a.k.a the CDH assumption, i.e., compute γ^{ab} given γ^a and γ^b in the transcript). For the security of key secrecy (cf. Section 2.4), it boils down to asking the adversary to distinguish between the actual key $K = \gamma^{ab}$ generated by the protocol and a random value $K' \in C_p$. This is exactly the Decisional Diffie–Hellman (DDH) assumption, which requires distinguishing between two distributions $D_0 := \{(\gamma, \gamma^a, \gamma^b, \gamma^{ab})\}$ and $D_1 := \{(\gamma, \gamma^a, \gamma^b, \gamma^c)\}$ for random a, b, c.

For our scheme, because of non-commutativity, the key exchange protocol requires more rounds, and hence the transcript will consist of many elements of the form $s, s*a, s*ab, \ldots$, etc. Therefore, a new hardness assumption is needed for the key secrecy proof to go through following [FGSW16]. Using the protocol based on metabelian groups in Section 1.2 as an example, we require that it is hard to distinguish between two distributions $\{(s,\ s*a,\ s*ab,\ s*aba^{-1},\ \ldots,\ s*aba^{-1}b^{-1}cdc^{-1},\ s*aba^{-1}b^{-1}cdc^{-1}d^{-1})\}$ and $\{(s,\ s*a,\ s*ab,\ s*aba^{-1},\ \ldots,\ s*aba^{-1}b^{-1}cdc^{-1},\ s*e)\}$ for random a, b, c, d, e. Note that here the shared key is $K = s*aba^{-1}b^{-1}cdc^{-1}d^{-1}$ and a random value is $s*e$. Besides that, we also need to consider DLP-like (Assumption 1) and CDH-like (Assumption 2) assumptions to ensure that the shared key cannot be obtained by the adversary from the transcript of the protocol.

Security support for key reusing. As mentioned, we use the symmetric group action underlying the monomial code equivalence problem, similar to [DKQ+25, Problem 2], for the concrete instantiation. However, this requires us to address the question of whether key reusing is secure under this group action. This is in particular of interest given that key reusing twice is not secure under the monomial group action [BCD+24b, BN25]. In Section 4.2, following [DKQ+25], we explain how our problem setting varies from that in [BCD+24b, BN25] and why their techniques do not apply to this symmetric group action formulation, at least not directly. In Section 4.3, we carry out cryptanalysis experiments based on Gröbner basis computations in Magma [BCP97]; see Table 1 for a summary. The experiment results support that reusing secret keys[3] in the context of the key exchange protocol is secure. It will be an interesting problem to carry out further cryptanalysis on Problem 4, as our experimental results also suggest that the current known techniques cannot help in attacking this natural reuse version of code equivalence.

1.5 Organization of the paper

In Section 2 we present preliminaries relevant to our discussions. In Section 3 we present our key exchange protocol for groups with laws, and prove its security. In Section 4 we recall the symmetric

[3] In fact, the key reusing in the key exchange protocol setting is a more restricted instance of key reusing in general, as seen in the comparison between Problem 3 and Problem 4 in Section 4.2.

group action underlying monomial code equivalence and present supporting evidence for the security of key using for this action. In Section 5 we present a concrete protocol by instantiating the key exchange protocol framework in Section 3 with the symmetric group action in Section 4.

2 Preliminaries

2.1 Notations

For $n \in \mathbb{N}$, $[n] := \{1, \ldots, n\}$. For a finite set S, $s \leftarrow\!\!\$\ S$ means that the element s is sampled uniformly and independently at random from S. We denote by $|S|$ the cardinality of S. An adversary is a probabilistic polynomial time (PPT) algorithm.

2.2 Linear algebra and some groups

Given a prime power q, let $\mathbb{F}_q$ be the field consisting of q elements. Denote by $\mathrm{Mat}(m \times n, q)$ the linear space of $m \times n$ matrices over $\mathbb{F}_q$, and $\mathrm{Mat}(n, q) := \mathrm{Mat}(n \times n, q)$. Let $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)$ be the set of matrices in $\mathrm{Mat}(m \times n, q)$ of rank $\min\{m, n\}$. Let $\mathbb{F}_q^n$ be the linear space of length-n row vectors over $\mathbb{F}_q$.

We denote by $\mathrm{GL}(n, q)$ the group of $n \times n$ invertible matrices over the field $\mathbb{F}_q$, and $\mathrm{D}(n, q)$ the group of $n \times n$ invertible diagonal matrices over $\mathbb{F}_q$. The symmetric group on $[n]$ is denoted by S_n. By interpreting $\sigma \in \mathrm{S}_n$ as a permutation matrix, we view S_n as a subgroup of $\mathrm{GL}(n, q)$. A matrix in $\mathrm{Mat}(n, q)$ is *monomial*, if it is a product of an invertible diagonal matrix and a permutation matrix. In other words, it is a matrix where each row and each column has exactly one non-zero entries. The group of monomial matrices is denoted by $\mathrm{Mon}(n, q)$.

2.3 Group action based cryptography

Basic definitions. Let G be a group and S a set. A left action of G on S is a map $\alpha : G \times S \to S$ satisfying the following properties: (i) $\alpha(\mathrm{id}, s) = s$ for all $s \in X$ and the identity element $\mathrm{id} \in G$; and (ii) $\alpha(g, \alpha(h, s)) = \alpha(gh, s)$ for all $g, h \in G$ and $s \in X$. A group action is said to be:

- *transitive* if for all $s, t \in S$, there exists $g \in G$ such that $\alpha(g, s) = t$;
- *faithful* if there does not exist $g \in G \backslash \{id\}$ such that $\alpha(g, s) = s$ for all $s \in S$, i.e., if $\alpha(g, s) = s$ for all $s \in S$ then $g = id$;
- *free* if whenever there exists $s \in S$ such that $\alpha(g, s) = s$ then $g = id$; and
- *regular* if it is free and transitive.

A right action of G on S can be defined similarly. For convenience, we may write $*$ for a right group action α, where $s * g = \alpha(g, s)$.

Given a group action α of G on S, the orbit of an element $s \in S$ is defined as $\mathsf{Orb}(s) := \{\alpha(g, s) : g \in G\}$. Note that if the group action is transitive, then $\mathsf{Orb}(s) = S$ for any $s \in S$. The stabilizer of s is defined by $\mathsf{Stab}(s) := \{g \in G : \alpha(g, s) = s)\}$ which is a subgroup of G. The Orbit-Stabilizer theorem says that for a finite group G, $|G| = |\mathsf{Stab}(s)| \cdot |\mathsf{Orb}(s)|$.

In this paper, we shall mostly consider finite groups acting on finite sets. To use group actions in algorithms, we assume that group and set elements have natural encodings, as well as group operations, group actions, and random samplings of group and set elements can be efficiently computed; see [BY90, JQSY19, AFMP20] for more details and certain variations.

Group actions in cryptography. Following [BY90], we say that a group action is *one-way*, if for a random s, the function $f_s : G \to S$ defined by $f_s(g) := \alpha(g, s)$ is one-way.[4]

The one-way assumption is closely related to the following algorithmic problem, known as the orbit problem, or the vectorization problem, or the Group Action Inverse Problem (GAIP).

Definition 1 (GAIP)**.** Given a group action $\alpha : G \times S \to S$, uniformly random $s \in S$, and uniformly random $t \in \mathsf{Orb}(s)$, find $g \in G$ such that $\alpha(g, s) = t$.

Another problem is called the *parallelization* problem, or the *Group Action Computational Diffie–Hellman (GACDH)* problem [MZ22], defined as follows.

Definition 2 (GACDH)**.** Given a group action $\alpha : G \times S \to S$, $s \in S$, $\alpha(g, s)$ and $\alpha(h, s)$ for uniformly-random $g, h \in G$, compute $\alpha(gh, s)$.

A third problem, known as the Group Action Decisional Diffie-Hellman (GADDH) or pseudo-random group actions [JQSY19, AFMP20], is defined as follows.

Definition 3 (GADDH))**.** A group action $\alpha : G \times S \to S$ is *pseudorandom*, if no PPT adversary can distinguish between the following distributions: (1) (s, t) for $s, t \leftarrow\!\!\$\ S$ and (2) (s, t) for $s \leftarrow\!\!\$\ S$ and $t \leftarrow\!\!\$\ \mathsf{Orb}(s)$.

Note that for efficiency improvements of many constructions based on group actions, we need to consider variants of the above problems for more instances; see [BKV19, BCD+24a] for details.

2.4 Key exchange protocols and security model

In this section, we follow [BR93, FGSW16] to define key exchange protocols and their security. We first describe the security model and then the security properties required for our key exchange protocols.

2.4.1 Security model

We consider two-party protocols which are defined by an interactive program Π that parties execute locally where parties belong to either a set of clients $\mathcal{C}$ or a set of servers $\mathcal{S}$. We set $\mathcal{I} = \mathcal{C} \cup \mathcal{S}$ to be the set of all identities in the system. Between invocations, the program maintains a state $\mathsf{st} = (\mathsf{crypt}, \mathsf{status}, \mathsf{role}, \mathrm{id}, \mathsf{pid}, \mathsf{sid}, \mathsf{key})$ where the components are defined as follows.

- $\mathsf{crypt} \in \{0, 1\}^*$ is some protocol-specific private session state used to maintain secret values from one invocation to the next. It will be set to the participants cryptographic keys upon initilization.
- $\mathsf{status} \in \{\mathsf{accept}, \mathsf{reject}, \perp\}$ is a variable that indicates the status of the key exchange phase. Initially set to $\perp$ to signify running, and may be changed to accept in accepted executions, or reject in runs in which the party rejects. We assume that once the status has been set to accept or reject, the value does not change anymore and the adversary immediately learns the value of status.

[4] In [BY90], the definition of one-way group actions is slightly different, in that the function f_s only needs to be one-way for a chosen s.

- $\mathsf{role} \in \{\mathsf{client},\mathsf{server}\}$ is the role of the participant. If $\mathrm{id} \in \mathcal{C}$ then $\mathsf{role} = \mathsf{client}$ and if $\mathrm{id} \in \mathcal{S}$ then $\mathsf{role} = \mathsf{server}$. [5]
- $\mathrm{id} \in \mathcal{I}$ is the identity of the party that owns this session.
- $\mathsf{pid} \in \mathcal{I} \cup \{\perp\}$ is the partner identifier initially $\mathsf{pid} = \perp$.
- $\mathsf{sid} \in \{0,1\}^* \cup \{\perp\}$ is the session identifier, initially set to $\perp$ and may be changed once to a non-trivial value, in which $\mathsf{status} = \mathsf{accept}$.
- $\mathsf{key} \in \{0,1\}^* \cup \{\perp\}$ is the key locally derived in this session, also called session key. Initially set to $\perp$, it may be changed once to a non-trivial value. If the key is different from $\perp$ then $\mathsf{status} = \mathsf{accept}$, and vice versa.

Formally, the program Π is a function which takes a state st as above, an input message m and returns a new state st' and a message m'. We write $(\mathsf{st}', m') \leftarrow \Pi(\mathsf{st}, m)$ for one step in the execution of the protocol that possesses message m.

We assume that the adversary $\mathcal{A}$ controls the communication network. In particular, the adversary $\mathcal{A}$ can make the queries to the following oracles.

- $\mathsf{NewSession}(i)$: this creates a globally unique administrative label l and a freshly initialized state st for the party; we denote the components of that session to be $l.\mathsf{crypt}$, $l.\mathsf{status}$, etc. This also sets $l.\mathrm{id} = i$. The adversary now can interact with this new session Π_i^l of the protocol Π via the label l. The adversary can specify $l.\mathsf{pid}$ upon session creation or leave it undefined.
- $\mathsf{Send}(l, m)$: executes $(\mathsf{st}', m') \leftarrow \Pi(l.\mathsf{st}, m)$, sets the local session state to st', and returns m' to the adversary. We also assume that adversary learns if the session changes its status $l.\mathsf{status}$ to accept or reject.
- $\mathsf{Corrupt}(i)$: the key sk_i is returned to the adversary and identity i is added to an initially empty set Corr of corrupted parties.
- $\mathsf{Reveal}(l)$: returns the session key $l.\mathsf{key}$.
- Test: this oracle is initialized with a secret bit $b \leftarrow\!\!\$\ \{0,1\}$. Upon querying the oracle with some label l it returns $\perp$ to the adversary if $l.\mathsf{status} \neq \mathsf{accept}$; otherwise it returns $l.\mathsf{key}$ if $b = 0$ or a fresh random key chosen from some distribution according to the protocol specification if $b = 1$. The task of the adversary is to determine b. We assume that the adversary can query the Test oracle only once.

Partnering. We say that two distinct sessions in an execution are *partnered* if their local session identifier variables have the same value different from $\perp$. Specifically, two sessions l and l' are partners if the predicate $\mathsf{Partners}(l, l')$ holds true, where

$$\mathsf{Partners}(l, l') = \mathsf{true} \Leftrightarrow [(l \neq l') \wedge (l.\mathsf{sid} = l'.\mathsf{sid} \neq \perp)].$$

[5] We follow the [BR93, FGSW16] client–server notation purely to distinguish the two ends of a two-party session; it carries no trust or capability difference in our symmetric protocol. In each session, the party that initiates the transcript (sends the first protocol message) is the client and the other party is the server. Either principal may adopt either role in different sessions.

Freshness. We say that a session with label l is *fresh* if the predicate $\mathsf{Fresh}(l)$ holds, i.e., if and only if the following conditions hold:

- $l.\mathsf{status} = \mathsf{accept}$
- The adversary has not issued a query $\mathsf{Reveal}(l)$ to the test session during the execution.
- $l.\mathrm{id} \notin \mathsf{Corr}$, i.e., the owner of the session has not been corrupted in the execution.
- For any l' that $\mathsf{Partners}(l, l')$ holds, the adversary has not issued a query to $\mathsf{Reveal}(l')$, i.e., no partner of the test session has been asked to reveal the session key.
- $l.\mathsf{pid} \notin \mathsf{Corr}$, i.e., the session has not been partnered with a party controlled by the adversary.

Authentication. Authentication guarantees that the identity of the partner for some session. We define the predicate $\mathsf{auth}(l, l')$ to be true if and only if $l.\mathsf{pid} = l'.\mathrm{id}$, i.e., session l authenticates the owner of session l' as it intended partner. Mutual authentication is expressed as $\mathsf{mauth}(l, l') = \mathsf{auth}(l, l') \wedge \mathsf{auth}(l', l)$.

2.4.2 Security properties

In this section, we recall the security properties for key exchange protocols. The first two traditional properties are *key secrecy* and *match security*. The last one is *key confirmation* which will ensure that after the execution of the protocol/session, the participants will have the same key.

Key secrecy. Key secrecy basically requires that (fresh) keys look random and are only available to the implicitly authenticated partners. We recall the experiment for key secrecy game as in [FGSW16] between a challenger and an adversary $\mathcal{A}$ as follows.
$\mathsf{Exp}^{\mathsf{secrecy}}_{\mathcal{A},\Pi}(\lambda)$:

- For each identity $i \in \mathcal{I}$ in the system, the challenger generates the secret key and public key for i and sends all $\{pk_i\}_{i\in\mathcal{I}}$ to the adversary $\mathcal{A}$.
- The challenger samples a random bit $b \leftarrow\!\!\$\ \{0, 1\}$.
- Adversary $\mathcal{A}$ is allowed to make queries to the oracles $\mathsf{NewSession}(.)$, $\mathsf{Send}(.)$, $\mathsf{Test}(b, .)$, $\mathsf{Corrupt}(.)$, $\mathsf{Reveal}(.)$.
- In the end, adversary $\mathcal{A}$ returns a guess b' for b.

We say that $\mathcal{A}$ wins the game if $b' = b$ and $\mathsf{Fresh}(l_{\mathsf{test}}) = \mathsf{True}$ where l_{test} is the session queried by the adversary $\mathcal{A}$ to $\mathsf{Test}(b, l_{\mathsf{test}})$. We say that a key exchange protocol Π provides *key secrecy* if for any PPT adversary $\mathcal{A}$ we have that

$$\Pr[\mathcal{A} \text{ wins}] \leq \frac{1}{2} + \mathrm{negl}(\lambda).$$

Match security. We define the predicate match which returns true if and only if all of the following properties hold.

(1) For all sessions l, l' with $\mathsf{Partners}(l, l') = \mathsf{true}$, it holds that $l.\mathsf{key} = l'.key$, i.e., partnered sessions derive the same key.

(2) For all sessions l, l', l'' with $\mathsf{Partners}(l, l') = \mathsf{true}$ and $\mathsf{Partners}(l, l'') = \mathsf{true}$, it holds that $l' = l''$, i.e., there is at most one partnered session for each session.

(3) For all sessions l, l' with $\mathsf{Partners}(l, l') = \mathsf{true}$, it holds that $l.\mathsf{role} \neq l'.\mathsf{role}$, i.e., two partnered sessions adopt the client-server relationship.

(4) For all l, l' with $\mathsf{Partners}(l, l') = \mathsf{true}$, it holds that $\mathsf{auth}(l, l') = \mathsf{true}$ or $l.\mathsf{pid} = *$, i.e., the partnered session has the intended owner where $l.\mathsf{pid} = *$ allows for an arbitrary owner.

We define the experiment $\mathsf{Exp}^{\mathsf{match}}_{\mathcal{A},\Pi}(\lambda)$ between the challenger and adversary $\mathcal{A}$ as similar to $\mathsf{Exp}^{\mathsf{secrecy}}_{\mathcal{A},\Pi}(\lambda)$ in which the adversary $\mathcal{A}$ does not allow to query the Test oracle. In particular, the experiment is as follows.

- For each identity $i \in \mathcal{I}$ in the system, the challenger generates the secret key and the public key for i and sends all $\{pk_i\}_{i\in\mathcal{I}}$ to the adversary $\mathcal{A}$.
- Adversary $\mathcal{A}$ is allowed to make queries to the oracles $\mathsf{NewSession}(.)$, $\mathsf{Send}(.)$, $\mathsf{Corrupt}(.)$, $\mathsf{Reveal}(.)$.
- In the end, Adversary returns b which is the evaluation of the predicate match on the resulting execution state.

We say that $\mathcal{A}$ wins if $b = \mathsf{false}$, that is, the adversary has managed to create a state that violates the security properties of the match. We say that a key exchange protocol provides Match security if for any PPT adversary $\mathcal{A}$, we have

$$\Pr[\mathcal{A} \text{ wins}] \leq \mathrm{negl}(\lambda).$$

Key confirmation. Key confirmation was developed by Fischlin et al. [FGSW16] to capture the property ensuring that the participating parties have obtained the same key after the execution of the key exchange protocol. There have been two notions for key confirmation in [FGSW16], that are *full key confirmation* which guarantees that when a party accepts a key, there exists some other party that has already accepted precisely that key, and *almost-full key confirmation* which ensures that when a party accepts a key, there is some session which, if it accepts, then it accepts the same key; see [FGSW16, Section III] for details. In this paper, we do not specify which key confirmation property is obtained for the protocol since in our protocol, both parties are sending the last messages; see Section 3 for details. What we care about is the generic transformation given in [FGSW16, Section V] that transforms a key exchange protocol to a key exchange protocol with key confirmation properties using MAC which we will recall briefly right now.

Let $\mathcal{M} = (\mathsf{KeyGen}, \mathsf{MAC}, \mathsf{Vf})$ be a key-only unforgeable MAC scheme, i.e., the adversary cannot output a message m^* with a valid MAC, without even being allowed to see a MAC for another message. Let KDF be a secure pseudorandom generator[6]. The transformation is given in Figure 1 as a blackbox that is applied to the key exchange protocol Π after both parties interact and return a key. After applying the blackbox transformation, the shared key between the parties will be key'.

[6] In [dSGFW20], KDF be a key derivation function which is assumed to be collision-resistant and their final key key' will be obtained by applying the KDF function to the session key key obtained from Π together with the transcripts generated in the sessions and the identities of both parties, i.e., $\mathsf{key}' = \mathsf{KDF}(\mathsf{key}, \mathsf{client}, \mathsf{server}, \mathsf{transcript}, \dots)$.

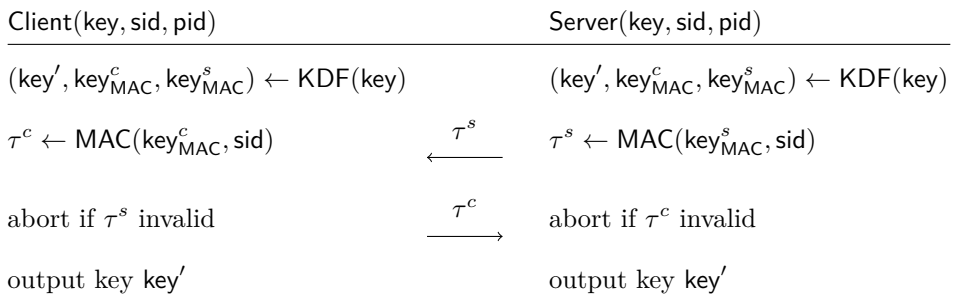

Figure 1: Blackbox for transforming a key exchange protocol Π to obtain a key exchange protocol Π_{MAC} with key confirmation. Here we assume that this transformation will be applied to Π after a session, i.e., after both parties obtain a key.

Theorem 1 ([FGSW16, Theorem 5.1])**.** *Let KDF be a secure pseudorandom generator. Suppose that (KeyGen, MAC, Vf) is a key-only unforgeable MAC scheme. If a key exchange Π has match security and key secrecy, then the transformed protocol Π_{MAC} in Figure 1 preserves those properties. In addition Π_{MAC} also provides key confirmation property. The protocol Π_{MAC} also provides the same authentication property as Π.*

In what follows, we will just describe our protocol Π and prove that the protocol Π provides two essential properties: match security and key secrecy. We then invoke the generic transformation in Figure 1 to obtain one with key confirmation.

3 Key exchange for actions by groups with laws

3.1 The key exchange protocol

Words and laws. We define words and laws in two variables. This is partly for convenience and also due to the fact that k-variable laws can be turned to 2-variable laws by a polynomial blow-up in length [BT19]. We note that k-variable laws could be more efficient in practice.

Definition 4. Let x and y be two non-commutative variables. A *word* w in x and y is a string in x, y, x^{-1}, and y^{-1}. For $a \in \mathbb{N}$, we shall write a sequence of a many x's (resp. x^{-1}'s) as x^a (resp. x^{-a}). The *length* of w is the number of alternations between x and y.

For example, a word w starting with x and ending with y is of the form $x^{a_1}y^{b_1}\dots x^{a_s}y^{b_s}$, where a_i, b_j are non-zero integers, and the length of w is $2s-1$.

Definition 5. A *law* in x and y is of the form $u = v$, where u and v are words in x and y. A (key-exchange) *useful* law is of the form $u = v$ in x and y where u ends with x and v ends with y.

Let G be a group. We say that G *satisfies the law* $u = v$, if for any assignment of x and y with g and h in G, the resulting group elements on the left and right sides are the same.

In the following, we will assume that laws are useful as above.

Key exchange protocol. Let G be a group and S be a set. Suppose G satisfies a useful law $u = v$, where $u = y^{b_1}x^{a_1}\dots y^{b_k}x^{a_k}$ and $v = x^{c_1}y^{d_1}\dots x^{c_\ell}y^{d_\ell}$ in variables x and y. Let $\alpha : G \times S \to S$ be a group action. Recall that from Section 2.3, we assume that the following can be computed in

polynomial time: group operations (multiplication and inverse), uniform sampling of G and S, and the group action function.

Consider the following key exchange protocol between Alice and Bob. To start with, they fix some $s_0 \in S$ as the public key. Then Alice randomly samples $g \leftarrow\!\!\$\, G$, and Bob randomly samples $h \leftarrow\!\!\$\, G$.

To obtain her secret key, Alice does the following. Recall that by our assumption, we have a word $u = y^{b_1}x^{a_1} \dots y^{b_k}x^{a_k}$, where $a_i, b_i \in \mathbb{Z}$, and all but possibly b_1 are non-zero. If $b_1 \neq 0$, then $k \geq 1$. If $b_1 = 0$, then $k \geq 2$.

1. Bob computes $t_1 := \alpha(h^{b_1}, s_0)$ and sends it to Alice.
2. For $i \in [k-1]$, do the following:
   (a) Alice computes $s_i := \alpha(g^{a_i}, t_i)$ and sends it to Bob.
   (b) Bob computes $t_{i+1} := \alpha(h^{b_{i+1}}, s_i)$ and sends it to Alice.
3. Alice computes $s_k = \alpha(g^{a_k}, t_k)$ as her secret key.

Bob can then obtain his secret key in the same way using v. This is just to note that v is of the form $x^{c_1}y^{d_1} \dots x^{c_\ell}y^{d_\ell}$, where $c_i, d_i \in \mathbb{Z}$, and all but possibly c_1 are non-zero. Specifically, Bob does the following.

1. Alice computes $s'_1 := \alpha(g^{c_1}, s_0)$ and sends it to Bob.
2. For $i \in [l-1]$, do the following:
   (a) Bob computes $t'_i := \alpha(h^{d_i}, s'_i)$ and sends it to Alice.
   (b) Alice computes $s'_{i+1} := \alpha(g^{c_{i+1}}, t'_i)$ and sends it to Bob.
3. Bob computes $t'_l := \alpha(h^{d_l}, s'_l)$ as his secret key.

After that, Alice and Bob need to run the Blackbox in Figure 1 to obtain key confirmation.

Key agreement. Upon the key confirmation procedure Blackbox in Figure 1, Alice's key agrees with Bob's. This is because Alice holds $\alpha(h^{b_1}g^{a_1} \dots h^{b_k}g^{a_k}, s_0)$ and Bob holds $\alpha(g^{c_1}h^{d_1} \dots g^{c_\ell}h^{d_\ell}, s_0)$. As $u = v$ is a law in G, we have that $h^{b_1}g^{a_1} \dots h^{b_k}g^{a_k} = g^{c_1}h^{d_1} \dots g^{c_\ell}h^{d_\ell}$, which ensures the key agreement.

The communication round number. The number of communications between Alice and Bob depends on the lengths of u and v. In the above illustration it is $2(k + \ell) - 2$.

3.2 Security proof

In this section, we provide the security proof of our key exchange protocol. For convenience, we first consider the first half transcript, as the second half transcript would follow the same reasoning. From the adversary Eve's viewpoint, she knows s_0 (the public information), $t_1 = \alpha(h^{b_1}, s_0)$, $s_1 = \alpha(g^{a_1}, t_1)$, $t_2 = \alpha(h^{b_2}, s_1)$, $\dots$, $t_k = \alpha(h^{b_k}, s_{k-1})$. To recover Alice's secret, she needs to be able to compute $s_k = \alpha(g^{a_k}, t_k)$. Note that $s_0, \dots, s_{k-1}$, $t_1, \dots, t_k$, and the exponents a_i and b_i, are known to Eve. So one approach for Eve to recover s_k is to compute g from the information $s_i = \alpha(g^{a_i}, t_i)$ for $i \in [k-1]$. The hardness of this is ensured by a DLP-like assumption, which we call Tuple Non-Commutative Discrete Logarithm Problem or TnDLP for short, defined as the following.

Assumption 1 (TnDLP)**.** Let $\alpha : G \times S \to S$ be a group action. Let $s_0 \in S$, $P \subseteq \mathbb{Z}$, and $(b_1, a_1, \ldots, b_k, a_k) \in P^{2k}$, and $(c_1, d_1, \ldots, c_\ell, d_\ell) \in P^{2\ell}$. For uniformly-random elements $g, h \in G$, no computationally bounded adversaries can compute g or h with non-negligible probability, given the following information:

1. $t_1 = \alpha(h^{b_1}, s_0)$, $s_1 = \alpha(g^{a_1}, t_1)$, $t_2 = \alpha(h^{b_2}, s_1)$, $\ldots$, $t_k = \alpha(h^{b_k}, s_{k-1})$, and
2. $s'_1 = \alpha(g^{c_1}, s_0)$, $t'_1 = \alpha(h^{d_1}, s'_1)$, $s'_2 = \alpha(g^{c_2}, t'_1)$, $\ldots$, $s'_\ell = \alpha(g^{c_\ell}, t'_{\ell-1})$.

Another approach is to compute the shared key $s_k = \alpha(g^{a_k}, t_k)$ given the $s_i = \alpha(g^{a_i}, t_i)$ and for $i \in [k-1]$ and $t_k = \alpha(h^{b_k}, s_{k-1})$ obtained from the transcript. This security is ensured by the hardness of the following CDH-like assumption, whose special case is called *weak unpredictable group action assumption* in [AFMP20]. We call this assumption to be Tuple Non-Commutative Computational Diffie-Hellman or TnCDH for short, defined as the following.

Assumption 2 (TnCDH)**.** Let $\alpha : G \times S \to S$ be a group action. Let $s_0 \in S$, $P \subseteq \mathbb{Z}$, and $(b_1, a_1, \ldots, b_k, a_k) \in P^{2k}$, and $(c_1, d_1, \ldots, c_\ell, d_\ell) \in P^{2\ell}$. For uniformly-random elements $g, h \in G$, no computationally bounded adversaries can compute $s_k := \alpha(h^{a_k}, t_k)$ or $t'_\ell := \alpha(h^{d_\ell}, s'_\ell)$ with non-negligible probability, given the following information:

1. $t_1 = \alpha(h^{b_1}, s_0)$, $s_1 = \alpha(g^{a_1}, t_1)$, $t_2 = \alpha(h^{b_2}, s_1)$, $\ldots$, $t_k = \alpha(h^{b_k}, s_{k-1})$, and
2. $s'_1 = \alpha(g^{c_1}, s_0)$, $t'_1 = \alpha(h^{d_1}, s'_1)$, $s'_2 = \alpha(g^{c_2}, t'_1)$, $\ldots$, $s'_\ell = \alpha(g^{c_\ell}, t'_{\ell-1})$.

Two assumptions TnDLP and TnCDH ensure that the shared key cannot be computed from the transcript in our key exchange protocol. Still, for the key secrecy property for our key exchange protocol, we need a DDH-like assumption, captured from the idea of the GACDH assumption, which we call Tuple Non-Commutative Decisional Diffie–Hellman Assumption or TnDDH for short, defined as the following.

Assumption 3 (TnDDH)**.** Given a group action $\alpha : G \times S \to S$, a fixed element $s_0 \in S$ and a finite set P of integer numbers. For $g, h \leftarrow\!\!\$\, G$, the probability distributions between

- $T_0 = \{(s_0, \alpha(h^{b_1}, s_0), \alpha(h^{b_1} g^{a_1}, s_0), \ldots, \alpha(h^{b_1} g^{a_1} \cdots g^{a_{k-1}} h^{b_k}, s_0), \alpha(h^{b_1} g^{a_1} \cdots h^{b_k} g^{a_k}, s_0))\}$ for $(b_1, a_1, \cdots, b_k, a_k) \leftarrow\!\!\$\, P^{2k}$; and
- $T_1 = \{(s_0, \alpha(h^{b_1}, s_0), \alpha(h^{b_1} g^{a_1}, s_0), \ldots, \alpha(h^{b_1} g^{a_1} \cdots g^{a_{k-1}} h^{b_k}, s_0), z)\}$ for $(b_1, a_1, \cdots, b_k) \leftarrow\!\!\$\, P^{2k-1}$ and $z \leftarrow\!\!\$\, \mathsf{Orb}(s_0)$

are computationally indistinguishable. Similarly, the probability distributions between

- $T'_0 = \{(s_0, \alpha(g^{c_1}, s_0), \alpha(g^{c_1} h^{d_1}, s_0), \ldots, \alpha(g^{c_1} h^{d_1} \cdots h^{d_{\ell-1}} g^{c_\ell}, s_0), \alpha(g^{c_1} h^{d_1} \cdots g^{c_\ell} h^{d_\ell}, s_0))\}$ for $(c_1, d_1, \cdots, c_\ell, d_\ell) \in P^{2\ell}$; and
- $T'_1 = \{(s_0, \alpha(g^{c_1}, s_0), \alpha(g^{c_1} h^{d_1}, s_0), \ldots, \alpha(g^{c_1} h^{d_1} \cdots h^{d_{\ell-1}} g^{c_\ell}, s_0), z)\}$ for $(c_1, d_1, \cdots, c_\ell) \in P^{2\ell-1}$ and $z \leftarrow\!\!\$\, \mathsf{Orb}(s_0)$

are computationally indistinguishable.

Note that TnDDH assumption is in fact a tuple variant of the pseudorandom group action assumption GADDH defined in Section 2.3.

3.2.1 Match security

It is clear from the protocol that the session fully determines the key (Property (1)). Properties (3) and (4) also easily follow. Finally, an honest party will contribute to a share in the orbit $\mathsf{Orb}(s_0)$, and so the probability of matching any other share is at most $n_s^2 \cdot \frac{1}{|\mathsf{Orb}(s_0)|}$, where n_s is the total number of sessions in the protocol, which is negligible (Property (2)).[7]

3.2.2 Key secrecy

The hardness of TnDDH assumption ensures the key secrecy of our key exchange protocol. Basically, it ensures that the adversary is unable to distinguish between the real session key and a random element. It is summarized in the following theorem.

Theorem 2. *Given a group action $\alpha : G \times S \to S$ that satisfies the TnDDH assumption (Assumption 3), our key exchange protocol described in Section 3.1 has key secrecy.*

Proof. Assume now that there is an adversary $\mathcal{A}$ against the key secrecy property that has a non-negligible advantage $\frac{1}{2} + \varepsilon$ in guessing correctly the value of random bit b chosen by the challenger in the experiment $\mathsf{Exp}^{\mathsf{secrecy}}_{\mathcal{A},\Pi}(\lambda)$. We will construct a distinguisher $\mathcal{D}$ to distinguish between two distributions T_0 and T_1. At the beginning, the input of $\mathcal{D}$ will be $(s_0, t_1^*, s_1^*, \ldots, t_k^*, z^*)$ which is withdrawn from either T_0 or T_1 with probability $\frac{1}{2}$. The distinguisher $\mathcal{D}$ works as follows.[8]

- Choose a random $l_{\mathsf{test}} \leftarrow\!\!\$\ \{1, \ldots, n_s\}$ where n_s is the upper bound of the number of sessions invoked by $\mathcal{A}$ in any interaction.
- Invoke $\mathcal{A}$ on a simulated interaction with Bob and Alice, provide $\mathcal{A}$ with public parameters $s_0, (b_1, a_1, \ldots, b_k, a_k)$.
- Whenever $\mathcal{A}$ activates a party to establish a new session $\mathsf{NewSession}(.)$ (except for the l_{test}-th session) or to receive a message, follow exactly as in the procedure of the key exchange protocol on behalf of the party. When a session is expired at a party, erase the corresponding session key from that party's memory. When a party is corrupted or a session is exposed, provide $\mathcal{A}$ all the information corresponding to that party or session as in a real interaction.
- When the l_{test}-th session, say (s, i) where $i = 1, \ldots, k$ is the step in the protocol, is invoked within Bob to exchange a key with Alice, let Bob sends the message (s, t_i) to Alice.
- When Alice is invoked to receive (s, t_j) from Bob, with $j = 1, \ldots, k-1$, let Alice sends (s, s_i) to Bob.
- If $\mathcal{A}$ chose the session s to be the test-session, provide $\mathcal{A}$ with z^* as the answer for the query.
- If the l_{test}-th session s is exposed or if a non l_{test}-th session is chosen as a test-session $\mathsf{Test}(b, .)$, or if $\mathcal{A}$ halts without choosing a test-session then $\mathcal{D}$ outputs a random bit $b' \leftarrow\!\!\$\ \{0, 1\}$.
- If $\mathcal{A}$ halts and outputs a guess b' for b, then $\mathcal{D}$ outputs b'.

[7] If the group action is transitive, the orbit size is equal to $|G|$, the order of the group G, which is normally large. If the group action is not transitive, especially the class of group actions of particular interest in ALTEQ [TDJ+22], MEDS [CNP+23], LESS [BMPS20], as well as in our instantiation in Section 5, the orbit size $|\mathsf{Orb}(s_0)|$ is normally large for a random s_0; see [BCD+24a, Section 6.1] for more details.

[8] Note that we just use the first half of the protocol, i.e., the case that only Alice has a shared key. It can be done similarly for the second half of the protocol in which Bob has a shared key.

It is easy to see that, if the input $(s_0, t_1^*, s_1^*, \ldots, t_k^*, z^*)$ of $\mathcal{D}$ is chosen from T_0 then $z^* = s_0 * h^{b_1} g^{a_1} \ldots h^{b_k} g^{a_k}$ which is the correct session key that Alice can obtain, following the procedure of the protocol. If the input of $\mathcal{D}$ is chosen from T_1 then the adversary $\mathcal{A}$ obtains a random value from the distribution of the keys generated by the protocol. Since the probability of choosing the input for $\mathcal{D}$ are equal, $\mathcal{D}$ outputs the same bit b' to the adversary $\mathcal{A}$, it follows that the advantage of $\mathcal{D}$ is the same with $\mathcal{A}$, which is $\frac{1}{2} + \varepsilon$ for non-negligible ε.

If l_{test}-th session is not the chosen test-session, with probability $1 - \frac{1}{n_s}$, then $\mathcal{D}$ outputs a random bit, hence the advantage of $\mathcal{D}$ is $\frac{1}{2}$. If l_{test}-th session is the chosen test-session, with probability $\frac{1}{n_s}$, the advantage of $\mathcal{D}$ is, as analyzed above, $\frac{1}{2} + \varepsilon$. Therefore, the advantage of $\mathcal{D}$ in distinguishing between T_0 and T_1 is

$$\frac{1}{2} \cdot \left(1 - \frac{1}{n_s}\right) + \left(\frac{1}{2} + \varepsilon\right) \cdot \frac{1}{n_s} = \frac{1}{2} + \frac{\varepsilon}{n_s},$$

which is non-negligible. □

Remark 1. With essentially the same proof, Theorem 2 also holds under a variant of Assumption 3 in which g and h are chosen to satisfy a specific law in group G, e.g., $(gh)^n = \mathrm{id}$. This will apply to our instantiation in Section 5; see Problem 5.

4 Secret key reusing for the symmetric group action underlying monomial code equivalence

4.1 Monomial code equivalence as a symmetric group action

Monomial code equivalence. A linear code over $\mathbb{F}_q$ is a subspace of $\mathbb{F}_q^n$. An m-dimensional code in $\mathbb{F}_q^n$ can be represented by a generator matrix $C \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$ with rows spanning the code. The monomial code equivalence problem is formally stated as follows.

Problem 1 (Monomial code equivalence (MCE))**.** For $n \in \mathbb{N}$, let $m \in [n]$. Let $C, C' \in \mathrm{Mat}(m \times n, q)$ be of rank m. Decide if there exist $A \in \mathrm{GL}(m, q)$ and $M \in \mathrm{Mon}(n, q)$, such that $ACM = C'$. If yes, compute such A and M.

Since a monomial matrix is a product of an invertible diagonal matrix and a permutation matrix, it is equivalent to formulate monomial code equivalence as asking if there exist $A \in \mathrm{GL}(m, q)$, $D \in \mathrm{D}(n, q)$, and an $n \times n$ permutation matrix P, such that $ACDP = C'$. By writing this way, we can introduce the symmetric group action from the following different perspectives.

Let $C, C' \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$, the set of full-rank matrices in $\mathrm{Mat}(m \times n, q)$. As mentioned, we can use $A \in \mathrm{GL}(m, q)$, $D \in \mathrm{D}(n, q)$, and $P \in \mathrm{S}_n$ to define equivalence between C and C'. As a result, there is more than one way to interpret the group action behind monomial code equivalence. Straightforwardly, it can be seen as the action of the group $\mathrm{GL}(m, q) \times (\mathrm{D}(n, q) \rtimes \mathrm{S}_n) = \mathrm{GL}(m, q) \times \mathrm{Mon}(n, q)$[9] acting on the set of generator matrices in $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. Additionally, from the viewpoint of coding theory, in line with [BMPS20, PS23, DDS24, BCD+24b, BN25, BEFN25, CPS25], it would be more natural to consider the monomial group $\mathrm{Mon}(n, q)$ acting on the set of m-dimensional codes in $\mathbb{F}_q^n$ instead of the generator matrices.

[9] Here $\rtimes$ denotes semidirect product of groups.

A symmetric group action underlying monomial code equivalence. Following [DKQ+25], we take a different perspective from above by formulating it as the symmetric group S_n acting on a set S consisting of equivalence classes. Specifically, for $C_1, C_2 \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$, we define an equivalence relation $\sim$ as $C_1 \sim C_2$ if and only if there exists some $A \in \mathrm{GL}(m, q)$ and $D \in \mathrm{D}(n, q)$ such that $C_1 = AC_2D$. This equivalence relation naturally partitions $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)$ into equivalence classes of $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)$ under the action of $\mathrm{GL}(m, q) \times \mathrm{D}(n, q)$. Note that this approach actually aligns with the natural viewpoint in that the monomial group $\mathrm{Mon}(n, q)$ acts on the set of m-dimensional codes in $\mathbb{F}_q^n$, as the codes can be treated as the equivalence classes of $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)$ under the action of $\mathrm{GL}(m, q)$.

Denote by $[C]_\sim := \{ACD : A \in \mathrm{GL}(m, q), D \in \mathrm{D}(n, q)\}$ the equivalence class determined by $\sim$ and corresponding to $C \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. Let $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim = \{[C]_\sim : C \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)\}$ be the set of equivalence classes under $\sim$. Now we wish to define an action of S_n on $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$. For $P \in \mathrm{S}_n$, since $[C]_\sim := \{ACD : A \in \mathrm{GL}(m, q), D \in \mathrm{D}(n, q)\}$ is a set of matrices, a natural map is to send $[C]_\sim$ to $[C]_\sim P := \{ACDP : A \in \mathrm{GL}(m, q), D \in \mathrm{D}(n, q)\}$. For S_n to act on $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$, we need to show that $[C]_\sim P$ is an element in $\mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$. This is ensured by the following proposition in [DKQ+25].

Proposition 3 ([DKQ+25, Proposition 1])**.** *Let $[C]_\sim \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$, $P \in \mathrm{S}_n$, and $[C]_\sim P$ be as above. Then $[C]_\sim P = [CP]_\sim$.*

Proposition 3 demonstrates that the map $\alpha : \mathrm{S}_n \times \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim \to \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$ by $P \in \mathrm{S}_n$ sending $[C]_\sim$ to $[C]_\sim P = [CP]_\sim$ is a well-defined group action.

Canonical form algorithm. At the end of our key exchange protocol, Alice and Bob each get C_1 and C_2 in the same equivalence class $[C]_\sim \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$. That is, there exist $A \in \mathrm{GL}(m, q)$ and $D \in \mathrm{D}(n, q)$ such that $C_1 = AC_2D$. To obtain a common key, Alice and Bob need to perform a *canonical form* algorithm to reach the same $C^* \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$. Canonical forms have been extensively explored for graphs and matrix tuples [Bab19, QS24]. Let G be a group acting on a set S. A canonical form algorithm takes an input $s \in S$ and returns a canonical representative $s^* \in \mathsf{Orb}(s) := \{g * s : \forall g \in G\}$. The term "canonical" means that for any input $s' \in \mathsf{Orb}(s)$, the output of the canonical form algorithm must be the same element $s^* \in \mathsf{Orb}(s)$. In our setting, we take $G = \mathrm{GL}(m, q) \times \mathrm{D}(n, q)$ and $S = \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. This particular problem has been well studied in [DMS24, DKQ+25]. We refer readers to the following result in [DKQ+25], which gives an algorithmic proof and strengthens [DMS24, Proposition 4] by eliminating the probability of failure.

Proposition 4 ([DKQ+25, Proposition 3])**.** *There is an $O(m^2 n)$-time canonical form algorithm for the action of $\mathrm{GL}(m, q) \times \mathrm{D}(n, q)$ on $\mathrm{Mat}(m \times n, q)$.*

4.2 Reusing secret keys

We have seen that the monomial code equivalence problem can be regarded as either a monomial group action problem or a symmetric group action problem. As a result, reusing secret keys leads to different problems in these two settings.

We first recall the following key reusing problem in the monomial group action setting, which has been studied in [DDS24, BCD+24b, BN25].

Problem 2 (t-samples monomial code equivalence)**.** For $n \in \mathbb{N}$, let $m \in [n]$. Let $\{C_i, C_i' : i \in [t]\} \subseteq \mathrm{Mat}(m \times n, q)$. Decide if there exist $A_i \in \mathrm{GL}(m, q)$ and $M \in \mathrm{Mon}(n, q)$, such that $A_i C_i M = C_i'$ for all $i \in [t]$. If yes, compute such a common monomial matrix M.

Accordingly, reusing secret keys for symmetric group actions gives rise to the following problem.

Problem 3 (t-samples symmetric action underlying monomial code equivalence)**.** For $n \in \mathbb{N}$, let $m \in [n]$. Let $\{C_i, C'_i : i \in [t]\} \subseteq \mathrm{Mat}(m \times n, q)$. Decide if there exist $A_i \in \mathrm{GL}(m, q)$, $\{D_i : i \in [n-1]\} \subseteq \mathrm{D}(n, q)$, and an $n \times n$ permutation matrix P, such that $A_i C_i D_i P = C'_i$ for all $i \in [t]$. If yes, compute such a common permutation matrix P.

In fact, for our key exchange protocol in Section 5, we need a special case of Problem 3, by setting $C'_i = C_{i+1}$.

Problem 4 (General diagonal-masked linear code equivalence (DmLCE))**.** For $n \in \mathbb{N}$, let $m \in [n]$. Let $\{C_i : i \in [n]\} \subseteq \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. Decide if there exist $\{A_i : i \in [n-1]\} \subseteq \mathrm{GL}(m, q)$, $\{D_i : i \in [n-1]\} \subseteq \mathrm{D}(n, q)$, and an $n \times n$ permutation matrix P, such that $A_i C_i D_i P = C_{i+1}$ for all $i \in [n-1]$. If yes, compute such a common permutation matrix P.

We note that Problem 2 reuses the same monomial group action from $\mathrm{Mon}(n, q)$ and changes only the general linear group action from $\mathrm{GL}(m, q)$ in each round. By contrast, Problem 3 and Problem 4 reuse the same symmetric group action from S_n, but keep changing the group action from a composite group $\mathrm{GL}(m, q) \times \mathrm{D}(n, q)$ to strengthen the hiding of secret information in each round.

To tackle Problem 2, [BCD+24b] proposed an efficient heuristic algorithm, which works when $t = 2$ samples are given for a code rate of $m/n = 1/2$. Later, [BN25] further refined it to solve Problem 2 with $t = 2$ samples for any code rate. The core idea of their algorithms is to construct a homogeneous linear system of equations where the common monomial matrix M is the only variable matrix. This is achieved by a standard attack using the parity-check matrices that generate the dual codes, which eliminates the need to consider the change-of-basis by A_i's on the generator matrices. In this way, since the number of equations increases quickly with the number of samples, while the number of variables always remains the same, this can be leveraged for a heuristic attack.

Interestingly, the approaches proposed in [BCD+24b, BN25] cannot be applied to tackle Problem 3 or Problem 4, at least not directly. This is because introducing the additional scalar variables of the D_i's and combining them with the common variable permutation matrix P would lead to a quadratic system of equations, which is essentially different from the linear system on which the heuristic analysis in [BCD+24b, BN25] is based. Moreover, their algorithms proceed by guessing entries of the common monomial matrix M—one entry at a time in [BCD+24b] and multiple entries at a time in [BN25]—and then testing whether setting an entry to 1 yields a valid solution. Note that for any monomial matrix, although its non-zero entries could have arbitrary non-zero values, it suffices to assume that one of the entries is 1 since the solution space of a homogeneous system includes all scalar multiples of the real solution. However, since the scaling matrix D_i keeps changing in Problem 3 and Problem 4, it would be either more costly or less effective to guess the scalar values than in Problem 2.

In the next subsection, we show the experiments for solving Problem 4 based on Gröbner basis computations, an essential tool in algebraic analysis, to provide supporting evidence of how the difficulties change as the system varies. Particularly, in Section 4.3.2, we consider a model that draws on the approaches in [BCD+24b, BN25], i.e., still using the parity-check matrices and guessing entries of the common permutation matrix, to verify the above reasoning and compare the results between the two settings of Problem 2 and Problem 4 (see Remark 4). Note that Problem 4 is the one actually used in the key exchange protocol in Section 5.

4.3 Experiments to support the security of multiple key uses

In this subsection, we present concrete experiments for solving DmLCE (i.e., Problem 4) based on the Gröbner basis methods.[10] This is in line with the common practice in [BFFP11, Bou11, BFV13, TDJ+22] and can serve as a supplementary work to [DKQ+25] as the security problems are analogous but they did not carry out cryptanalysis experiments to support the conjectured hardness. Now we introduce two main models for the construction of a polynomial system to be solved by Gröbner basis methods.

4.3.1 Direct modeling

Given $\{C_i : i \in [n]\} \subseteq \mathrm{Mat}(m \times n, q)$, we set $(n-1)$ many $d \times d$ variable matrices $\{A_i : i \in [n-1]\}$, $2(n-1)$ many n-variable diagonal matrices $\{D_i, D_i^{-1} : i \in [n-1]\}$, and an $n \times n$ variable matrix P, satisfying the following equations (where I_n is the $n \times n$ identity matrix, and $P_{i,j}$ refer to the (i,j)th entry of P, respectively):

$$\begin{cases} A_i C_{i+1} = C_i D_i P \text{ for all } i \in [n-1] & (2) \\ D_i D_i^{-1} = I_n \text{ for all } i \in [n-1] & (3) \\ D_i^{-1} D_i = I_n \text{ for all } i \in [n-1] & (4) \\ P_{i,j}(P_{i,j} - 1) = 0 \text{ for all } i,j \in [n] & (5) \\ P_{i,j} P_{i,j'} = 0 \text{ for all } i,j,j' \in [n] & (6) \\ P_{i,j} P_{i',j} = 0 \text{ for all } i,i',j \in [n] & (7) \\ \sum_{j \in [n]} P_{i,j} = 1 \text{ for all } i \in [n] & (8) \\ \sum_{i \in [n]} P_{i,j} = 1 \text{ for all } j \in [n] & (9) \end{cases}$$

We now examine these equations one by one. Eqs. (2) are direct transformations of the original equations to reduce the polynomial degree from cubic to quadratic. Eqs. (3) and (4) are the constraint equations that enforce the invertibility of D_i's. Note that we do not introduce additional variable matrices to ensure A_i's and P are invertible because this can be implied by the current equations, and increasing variables may slow down the Gröbner basis computation. Eqs. (5) to (9) impose the constraint that P is a permutation matrix. Specifically, Eq. (5) means that each (i,j)th entry of P must be either 0 or 1. Eqs. (6) and (7) means the product of any two entries in the same row or column is 0, since each row and column contains at most one entry equal to 1. Eqs. (8) and (9) further guarantee that each row and column contains at least one non-zero entry.

Remark 2. Eq. (3) and Eq. (4) seem to be redundant duplicates in the polynomial system. However, we add both of them because, empirically, the performance of the Gröbner basis attack can be improved by increasing the number of equations while keeping the number of variables and the polynomial degree unchanged.

Given this setting and using Magma [BCP97], we implemented two versions of attacks to the DmLCE problem.

[10] Our experiments were conducted on a machine equipped with an Apple M2 Pro CPU (10 cores, 10 threads) and 16 GB of 6400 MT/s unified LPDDR5 SDRAM (128-bit bus, 200 GB/s bandwidth) The full codes are available at https://github.com/AnonymousSubmission-crypto/DmLCE_solver/

The first one is directly solving the Gröbner basis of the above polynomials. For the results, we carried out successful computations for $m = 3$, $n = 6$ and $q = 131$ in about 539 seconds, but could not achieve a successful one for $m = 4$, $n = 8$ and $q = 131$ within nine hours, reflecting the drastically increased complexity as m and n grow.

The second approach is adding a *guess* for the position of the 1 in the first row and the first column of the variable matrix P. This is a common technique in attacks based on Gröber basis, and in this case, it adds a multiplicative cost which is polynomial in n. Note that such a guess would determine the entire first row and first column, since each row and column of P contains only one 1 while all others are 0, thereby $(2n-1)$ many variables would be eliminated in this way. For the results, we carried out successful computations in about 13 seconds for $m = 4$, $n = 8$ and $q = 131$, and in about 11835 seconds for $m = 5$, $n = 10$ and $q = 131$, but could not achieve a successful one for $m = 6$, $n = 12$ and $q = 131$ within nine hours, reflecting again the drastically increased complexity as m and n grow.

4.3.2 Parity-check modeling

As seen in [BBMP24, BCD+24b, BN25, BEFN25], one can avoid setting additional variables for invertible matrices $\{A_i : i \in [n-1]\}$ by computing their *parity-check* matrices, which reduces a large number of variables in the polynomial system for Gröbner basis computation. Nevertheless, we note that this approach may not always outperform the direct method, as the total number of equations also gets reduced, which is indeed reflected by our experimental results.

To compute the parity-check matrices, we first need to put the linear codes in their *systematic form*. For any generator matrix $C \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$, there always exist an invertible matrix $A \in \mathrm{GL}(m, q)$ and a $n \times n$ permutation matrix R to convert C into the following form:

$$\mathsf{SystematicForm}(C) = ACR = (I_m \mid M) = \begin{pmatrix} 1 & 0 & \cdots & 0 & * & * & \cdots & * \\ 0 & 1 & \cdots & 0 & * & * & \cdots & * \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & * & * & \cdots & * \end{pmatrix} \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q),$$

where I_m refers to the $m \times m$ identity matrix and $M \in \mathrm{Mat}(m \times (n-m), q)$. More specifically, AC corresponds to the reduced row echelon form of C, and R can be efficiently computed by iterating through all the column vectors of AC to find each standard basis vector as they first occur and rearranging them to form the left block I_m while leaving the remaining entries to form the right block M.

With the notation above, the parity-check matrix of C is defined to be

$$\mathsf{ParityCheck}(C) = (-M^{\mathrm{t}} \mid I_{n-m})R^{\mathrm{t}} \in \mathrm{Mat}_{\mathrm{f}}((n-m) \times n, q).$$

Since A^{t} is invertible, it is straightforward to verify that

$$(-M^{\mathrm{t}} \mid I_{n-m}) \cdot (I_m \mid M)^{\mathrm{t}} = \mathsf{ParityCheck}(C) \cdot C^{\mathrm{t}} A^{\mathrm{t}} = \mathsf{ParityCheck}(C) \cdot C^{\mathrm{t}} = 0 \in \mathrm{Mat}((n-m) \times m, q).$$

Therefore, we can use the parity-check matrices to rewrite the equation $A_i C_{i+1} = C_i D_i P$ as $\mathsf{ParityCheck}(C_{i+1}) \cdot (C_i D_i P)^{\mathrm{t}} = 0$ and thus avoid setting the invertible matrices A_i's as variables.

Remark 3. Given a generator matrix $C_0 \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. For any $C \in \{AC_0 : A \in \mathrm{GL}(m, q)\}$, we have $\mathsf{ParityCheck}(C_0) \cdot C^{\mathrm{t}} = 0$ by definition. This leads to a natural idea to find a similar

function $\mathsf{DiagParityCheck} : \mathrm{Mat}_{\mathrm{f}}(m \times n, q) \to \mathrm{Mat}_{\mathrm{f}}(k \times n, q)$ for some $k \in \Theta(n)$ such that for any $C \in [C_0]_\sim = \{AC_0D : A \in \mathrm{GL}(m, q), D \in \mathrm{D}(n, q)\}$, we can get $\mathsf{DiagParityCheck}(C_0) \cdot C^{\mathrm{t}} = 0$. If such a function exists, one could rewrite the equation $A_iC_{i+1} = C_iD_iP$ as

$$\mathsf{DiagParityCheck}(C_{i+1}) \cdot (C_iP)^{\mathrm{t}} = 0,$$

leaving the common permutation matrix P as the only variable matrix and thereby reducing Problem 4 to a special case of Problem 2. However, we confirm that such $\mathsf{DiagParityCheck}$ does NOT exist in general by the following simple example over $\mathbb{F}_q$ for any $q > 2$:

$$A = I_2, \quad C_0 = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}, \quad D_1 = I_3, \quad D_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & q-1 \end{pmatrix}.$$

Picking an arbitrary row of $\mathsf{DiagParityCheck}(C_0)$, denoted by $\begin{pmatrix} c_1 & c_2 & c_3 \end{pmatrix}^{\mathrm{t}} \in \mathbb{F}_q^3$, it follows that

$$\begin{pmatrix} c_1 & c_2 & c_3 \end{pmatrix} \cdot (AC_0D_1)^{\mathrm{t}} = 0 \iff \begin{cases} c_1 + c_3 = 0 \\ c_2 + c_3 = 0 \end{cases}$$

and

$$\begin{pmatrix} c_1 & c_2 & c_3 \end{pmatrix} \cdot (AC_0D_2)^{\mathrm{t}} = 0 \iff \begin{cases} c_1 - c_3 = 0 \\ c_2 - c_3 = 0 \end{cases}$$

In this case, it must hold that $c_1 = c_2 = c_3 = 0$, implying $\mathsf{DiagParityCheck}(C_0) = 0$.

Now given $\{C_i : i \in [n]\} \subseteq \mathrm{Mat}(m \times n, q)$ again, we need to set $(n-1)$ many n-variable diagonal matrices $\{D_i : i \in [n-1]\}$, and an $n \times n$ variable matrix P, satisfying the following equations (where Eqs. (11) to (15) just remain the same as Eqs. (5) to (9) before, ensuring that P is a permutation matrix):

$$\begin{cases} \mathsf{ParityCheck}(C_{i+1}) \cdot (C_iD_iP)^{\mathrm{t}} = 0 \text{ for all } i \in [n-1] & (10) \\ P_{i,j}(P_{i,j} - 1) = 0 \text{ for all } i, j \in [n] & (11) \\ P_{i,j}P_{i,j'} = 0 \text{ for all } i, j, j' \in [n] & (12) \\ P_{i,j}P_{i',j} = 0 \text{ for all } i, i', j \in [n] & (13) \\ \sum_{j \in [n]} P_{i,j} = 1 \text{ for all } i \in [n] & (14) \\ \sum_{i \in [n]} P_{i,j} = 1 \text{ for all } j \in [n] & (15) \end{cases}$$

Note that in the above polynomial system, the variable number is $(n-1)n + n^2 = O(n^2)$, and the equation number is $(n-1) \cdot (n-m) \cdot n + O(n^3)$, which for $m = c \cdot n$ is $O(n^3)$. As the number of equations is fewer than $O(n^4)$, a direct application of the linearization techniques for solving quadratic polynomial systems by Kipnis and Shamir [KS99] would not work, at least not directly.

Similar to the direct modeling, we implemented two versions of attacks to the DmLCE problem in Magma [BCP97].

The first one is directly solving the Gröbner basis of the above polynomials. For the results, we carried out successful computations for $m = 3$, $n = 6$ and $q = 131$ in about 68 seconds, but could not achieve a successful one for $m = 4$, $n = 8$ and $q = 131$ within nine hours, reflecting the drastically increased complexity as m and n grow.

The second approach is still adding a guess for the position of the 1 in the first row and the first column of the variable matrix P. Again, such a guess would determine the entire first row and first column, since each row and column of P contains only one 1 while all others are 0, thereby $(2n-1)$ many variables would be eliminated in this way. For the results, we carried out successful computations in about 20 seconds for $m = 4$, $n = 8$ and $q = 131$, but could not achieve a successful one for $m = 5$, $n = 10$ and $q = 131$ within nine hours, reflecting again the drastically increased complexity as m and n grow. We summarize the experimental results for both direct modeling and parity-check modeling into Table 1.

q	m	n	Direct method	Direct method with guessing	Parity-check method	Parity-check method with guessing
131	3	6	538.72 seconds	0.080 seconds	68.143 seconds	0.050 seconds
131	4	8	>9 hours	13.410 seconds	>9 hours	19.780 seconds
131	5	10	>9 hours	11835.160 seconds ($\sim$3 hours)	>9 hours	>9 hours
131	6	12	>9 hours	>9 hours	>9 hours	>9 hours

Table 1: A summary of experimental results for two models, and with/without guessing one row and one column in P.

Remark 4. According to [BCD+24b, Table 3], their algorithm can solve Problem 2 with $t = 2$ samples for $m = 64$ and $n = 128$ within around six hours. This is a sharp contrast to our experimental results in Table 1, supporting that the techniques do not directly carry over in attacking Problem 4.

Some brief remarks on the experimental results in Table 1 are due. First, the time in the cells represents only the Gröbner basis computation time. Other steps, such as constructing the polynomial system of equations, are efficient, so their running times are negligible in comparison. Second, when it comes to '>9 hours', we mean that the experiment did not finish within 9 hours. In these cases, we also found that the step degrees in the Gröbner basis computations were mostly higher than in experiments with smaller n, supporting the complexity grows with n. Third, as is well known in Gröbner basis computations, the order of the underlying field q has only a marginal effect on computation time; the dominant factors are the number of variables, the degrees of the polynomials, and the chosen monomial orders.

5 An instantiation based on monomial code equivalence

In this section, we present a concrete key exchange protocol following the framework in Section 3 and based on the symmetric group action underlying monomial code equivalence in Section 4. Before that, we review the monomial code equivalence problem in terms of symmetric group actions, and then illustrate our idea of using a law of the form $(PQ)^{\lceil n/2 \rceil} = (Q^{-1}P^{-1})^{\lfloor n/2 \rfloor}$ for P, Q in the symmetric group S_n.

5.1 The key exchange protocol based on monomial code equivalence

We have seen the interpretation of monomial code equivalence as a symmetric group action. To prepare for our key exchange protocol based on this group action, we recall the following from symmetric groups. Let S_n be the symmetric group on $[n]$. For distinct $i_1, \ldots, i_k \in [n]$, denote by $(i_1, \ldots, i_k)$ the permutation sending i_1 to i_2, $\ldots$, i_{k-1} to i_k, and i_k to i_1, and leaving others fixed. Recall that every permutation admits a cycle decomposition. A permutation is called a *full-cycle*, if $P = (1, i_1, \ldots, i_{n-1})$ where $i_1, \ldots, i_{n-1} \neq 1$ are distinct. Note that a full-cycle permutation is of

order n, i.e., $P^n = \mathrm{id}$. If the product PQ of two permutations P and Q is a full-cycle, then by $(PQ)^n = \mathrm{id}$, we have a law of the form $(PQ)^{\lceil n/2 \rceil} = (Q^{-1}P^{-1})^{\lfloor n/2 \rfloor}$. In particular, if n is prime, then P being a full-cycle is equivalent to $P^n = \mathrm{id}$.

Fact 1. *Let P and Q be uniformly randomly sampled permutations from S_n. The probability that PQ forms a full-cycle is $1/n$.*

We can then describe the key exchange protocol based on this symmetric group action in Algorithm 1. Since we are only interested in the case when PQ forms a full-cycle such that $(PQ)^n = \mathrm{id}$ where P, Q are uniformly randomly sampled permutations from S_n, to avoid the cases when PQ's order is a divisor of n, we simply choose n to be prime. Though, for composite n, it actually just takes a low risk according to [NP07, Theorem 1.1]: for sufficiently large n and a uniformly randomly sampled permutation $P \in \mathrm{S}_n$, the probability that $P^n = \mathrm{id}$ is $\frac{1}{n} + \frac{c}{n^2} + O(\frac{1}{n^{2.5-o(1)}})$ where $c = 0$ if n is odd and $c = 2$ if n is even. This shows that, given $P^n = \mathrm{id}$, the probability that P has order dividing n is much smaller than the probability that P has order exactly n.

Key exchange procedure.

Input:

Public: The action $\alpha : \mathrm{S}_n \times \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim \to \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$, and $C_{\mathrm{public}} \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$, with $m \leq n$ and n is an odd prime.

Output: Key pair $C_n, C'_n \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$ satisfying $C_n \sim C'_n$.

1 Alice randomly samples $P \leftarrow\!\!\$\ \mathrm{S}_n$, $A_0 \in \mathrm{GL}(m, q)$ and $D_0 \in \mathrm{D}(n, q)$, then she computes $C_1 \leftarrow A_0 C_{\mathrm{public}} D_0 P$ and sends C_1 to Bob.

2 Bob randomly samples $Q \leftarrow\!\!\$\ \mathrm{S}_n$.

3 **for** $i \in [\frac{n+1}{2} - 1]$ **do**

4 Bob randomly samples $A_{2i-1} \in \mathrm{GL}(m, q)$ and $D_{2i-1} \in \mathrm{D}(n, q)$, then he computes $C_{2i} \leftarrow A_{2i-1} C_{2i-1} D_{2i-1} Q$ and sends C_{2i} to Alice.

5 Alice randomly samples $A_{2i} \in \mathrm{GL}(m, q)$ and $D_{2i} \in \mathrm{D}(n, q)$, then she computes $C_{2i+1} \leftarrow A_{2i} C_{2i} D_{2i} P$ and sends C_{2i+1} to Bob.

6 **end**

7 Bob randomly samples $A_n \in \mathrm{GL}(m, q)$ and $D_n \in \mathrm{D}(n, q)$, then he computes $C_{n+1} \leftarrow A_n C_n D_n Q$ and computes the canonical form of C_{n+1} as in Proposition 4. `This settles one key in the key pair.`

8 Bob randomly samples $A'_0 \in \mathrm{GL}(m, q)$ and $D'_0 \in \mathrm{D}(n, q)$, then he computes $C'_1 \leftarrow A'_0 C_{\mathrm{public}} D'_0 Q^{-1}$ and sends C'_1 to Alice.

9 **for** $i \in [\frac{n-1}{2} - 1]$ **do**

10 Alice randomly samples $A'_{2i-1} \in \mathrm{GL}(m, q)$ and $D'_{2i-1} \in \mathrm{D}(n, q)$, then she computes $C'_{2i} \leftarrow A'_{2i-1} C'_{2i-1} D'_{2i-1} P^{-1}$ and sends C'_{2i} to Bob.

11 Bob randomly samples $A'_{2i} \in \mathrm{GL}(m, q)$ and $D'_{2i} \in \mathrm{D}(n, q)$, then he computes $C'_{2i+1} \leftarrow A'_{2i} C'_{2i} D'_{2i} Q^{-1}$ and sends C'_{2i+1} to Alice.

12 **end**

13 Alice randomly samples $A'_{n-2} \in \mathrm{GL}(m, q)$ and $D'_{n-2} \in \mathrm{D}(n, q)$, then she computes $C'_{n-1} \leftarrow A'_{n-2} C'_{n-2} D'_{n-2} P^{-1}$ and computes the canonical form of C'_{n-1} as in Proposition 4. `This settles another key in the key pair.`

14 Apply the transformation Blackbox in Figure 1 for the key to be the canonical form of C_{n+1} for Bob and the canonical form of C'_{n-1} for Alice. If it aborts, then return to Step 1.

Remark 5. In Algorithm 1, we expect that Alice's permutation $P \in \mathrm{S}_n$ and Bob's permutation $Q \in \mathrm{S}_n$ satisfy that $(PQ)^n = \mathrm{id}$. To obtain such P and Q, Alice and Bob each need to uniformly

sample a random permutation from S_n (in Steps 1 and 2 respectively), and the probability of $(PQ)^n = \mathrm{id}$ for prime n is $1/n$ by Fact 1. At Step 14, if $(PQ)^n \neq \mathrm{id}$, then the key confirmation will not succeed, causing it to abort and return to Step 1 so that Alice and Bob can re-sample different P and Q. The algorithm will repeat until $(PQ)^n = \mathrm{id}$.

Remark 6. The use of full-cycle premutations raises the question as to if the monomial code equivalence problem would be easier if the underlying permutation is a full-cycle. However, this is not the case, because we can reduce from general permutations to full-cycles by a random reduction: if C_1 and C_2 are related by a permutation Q, we can apply a random permutation R to C_2 to get C_3, so with probability $1/n$, C_1 and C_3 are related by a full-cycle.

Proposition 5 (Correctness of the key exchange protocol)**.** *There exist* $A \in \mathrm{GL}(m, q)$ *and* $D \in \mathrm{D}(n, q)$ *such that* $AC_{n+1}D = C'_{n-1}$.

Proof. Before proving $C_{n+1} \sim C'_{n-1}$, we show the commutativity of permutation matrices and diagonal matrices. Let $\sigma \in \mathrm{S}_n$ and encode it as an $n \times n$ permutation matrix P, and let $D \in \mathrm{D}(n, q)$. Denote by D_i the ith diagonal entry of D. It is straightforward to verify that $D' := P^{-1}DP$ is still a diagonal matrix whose ith diagonal entry is $D_{\sigma(i)}$. This gives us that there always exists some $D' \in \mathrm{D}(n, q)$ such that $DP = PP^{-1}DP = PD'$ for any given $n \times n$ permutation matrix P and $D \in \mathrm{D}(n, q)$.

Now following the protocol, we have that

$$\begin{aligned} C_{n+1} &= \prod_{i=0}^{n} A_i \cdot C_{\text{public}} \cdot \prod_{i=0}^{\frac{n+1}{2}-1} (D_{2i} P D_{2i+1} Q) \\ &= \prod_{i=0}^{n} A_i \cdot C_{\text{public}} \cdot (PQ)^{\frac{n+1}{2}} \cdot \prod_{i=0}^{\frac{n+1}{2}-1} \left(\tilde{D}_{2i} \tilde{D}_{2i+1}\right) \end{aligned}$$

for some $\{\tilde{D}_i : i \in [n]\} \subseteq \mathrm{D}(n, q)$. Note that $\prod_{i=0}^{n} A_i \in \mathrm{GL}(m, q)$ and $\prod_{i=0}^{\frac{n+1}{2}-1} (\tilde{D}_{2i} \tilde{D}_{2i+1}) \in \mathrm{D}(n, q)$. This ensures that $C_{n+1} \sim C_{\text{public}} (PQ)^{\frac{n+1}{2}}$.

On the other hand, we have that

$$\begin{aligned} C'_{n-1} &= \prod_{i=0}^{n-2} A'_i \cdot C_{\text{public}} \cdot \prod_{i=0}^{\frac{n-1}{2}-1} \left(D'_{2i} Q^{-1} D'_{2i+1} P^{-1}\right) \\ &= \prod_{i=0}^{n-2} A'_i \cdot C_{\text{public}} \cdot (Q^{-1}P^{-1})^{\frac{n-1}{2}} \cdot \prod_{i=0}^{\frac{n-1}{2}-1} \left(\tilde{D}'_{2i} \tilde{D}'_{2i+1}\right) \end{aligned}$$

for some $\{\tilde{D}'_i : i \in [n-2]\} \subseteq \mathrm{D}(n, q)$. Note that $\prod_{i=0}^{n-2} A'_i \in \mathrm{GL}(m, q)$ and $\prod_{i=0}^{\frac{n-1}{2}-1} (\tilde{D}'_{2i} \tilde{D}'_{2i+1}) \in \mathrm{D}(n, q)$. This ensures that $C'_{n-1} \sim C_{\text{public}} (Q^{-1}P^{-1})^{\frac{n-1}{2}}$.

Since $(PQ)^n = I_n$, we have a law of the form $(PQ)^{\lceil n/2 \rceil} = (Q^{-1}P^{-1})^{\lfloor n/2 \rfloor}$, which implies C_{n+1} and C'_{n-1} are equivalent to the same matrix under the equivalence relation $\sim$. By transitivity, we can conclude that $C_{n+1} \sim C'_{n-1}$. □

The communication round number. It is expected to have $(2n^2 - 2n)$ many communication rounds, since the probability of obtaining a full-cycle permutation is $1/n$ and hence we may need to repeat the key exchange n many times.

5.2 The security assumption

The key secrecy property of Algorithm 1 depends on Theorem 2, which in turn relies on Assumption 3. Instantiating a variant of Assumption 3 with the symmetric group action $\alpha : \mathrm{S}_n \times \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim \to \mathrm{Mat}_{\mathrm{f}}(m \times n, q)/\sim$, the assumed hardness of the following problem would give us the key secrecy of Algorithm 1, by Theorem 2 and Remark 1.

Problem 5. For odd prime $n \in \mathbb{N}$, let $m \in [n]$, distinguish each pair of the following distributions:

1(a) Let $P, Q \leftarrow\!\$\ \mathrm{S}_n$ such that $(PQ)^n = I_n$. For $i \in [n]$, let $A_i \leftarrow\!\$\ \mathrm{GL}(m, q)$ and $D_i \leftarrow\!\$\ \mathrm{D}(n, q)$. Let $C_0 \leftarrow\!\$\ \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. The pseudorandom distribution gives $(C_0, C_1 = A_0 C_0 D_0 P, C_2 = A_1 C_1 D_1 Q, C_3 = A_2 C_2 D_2 P, \ldots, C_n = A_{n-1} C_{n-1} D_{n-1} P, C_{n+1} = A_n C_n D_n Q)$.

(b) Let $P, Q \leftarrow\!\$\ \mathrm{S}_n$ such that $(PQ)^n = I_n$. For $i \in [n-1]$, let $A_i \leftarrow\!\$\ \mathrm{GL}(m, q)$ and $D_i \leftarrow\!\$\ \mathrm{D}(n, q)$. Let $C_0, C_{\mathrm{random}} \leftarrow\!\$\ \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. The random distribution gives $(C_0, C_1 = A_0 C_0 D_0 P, C_2 = A_1 C_1 D_1 Q, C_3 = A_2 C_2 D_2 P, \ldots, C_n = A_{n-1} C_{n-1} D_{n-1} P, C_{\mathrm{random}})$.

2(a) Let $P, Q \leftarrow\!\$\ \mathrm{S}_n$ such that $(PQ)^n = I_n$. For $i \in [n-2]$, let $A'_i \leftarrow\!\$\ \mathrm{GL}(m, q)$ and $D'_i \leftarrow\!\$\ \mathrm{D}(n, q)$. Let $C_0 \leftarrow\!\$\ \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. The pseudorandom distribution gives $(C_0, C'_1 = A'_0 C'_0 D'_0 Q^{-1}, C'_2 = A'_1 C'_1 D'_1 P^{-1}, C'_3 = A'_2 C'_2 D'_2 Q^{-1}, \ldots, C'_{n-2} = A'_{n-3} C'_{n-3} D'_{n-3} Q^{-1}, C'_{n-1} = A'_{n-2} C'_{n-2} D'_{n-2} P^{-1})$.

(b) Let $P, Q \leftarrow\!\$\ \mathrm{S}_n$ such that $(PQ)^n = I_n$. For $i \in [n-3]$, let $A'_i \leftarrow\!\$\ \mathrm{GL}(m, q)$ and $D'_i \leftarrow\!\$\ \mathrm{D}(n, q)$. Let $C_0, C_{\mathrm{random}} \leftarrow\!\$\ \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. The random distribution gives $(C_0, C'_1 = A'_0 C'_0 D'_0 Q^{-1}, C'_2 = A'_1 C'_1 D'_1 P^{-1}, C'_3 = A'_2 C'_2 D'_2 Q^{-1}, \ldots, C'_{n-2} = A'_{n-3} C'_{n-3} D'_{n-3} Q^{-1}, C_{\mathrm{random}})$.

To understand Problem 5, it would be instructive to study its computational version, Problem 4, and the following problem. Please refer back to Section 4.2 and Section 4.3 for discussions on the hardness evidence of our reuse version of code equivalence. We also leave further cryptanalysis on this as an interesting open problem, as our experimental results suggest that the current known techniques do not contribute to attacking these natural variants from code equivalence.

Problem 6 (2-samples diagonal-masked linear code equivalence (2-DmLCE))**.** For odd prime $n \in \mathbb{N}$, let $m \in [n]$. Let $C_1, C_2, C_3 \in \mathrm{Mat}_{\mathrm{f}}(m \times n, q)$. Decide if there exist $A_1, A_2 \in \mathrm{GL}(m, q)$, $D_1, D_2 \in \mathrm{D}(n, q)$, and an $n \times n$ permutation matrix P, such that $A_1 C_1 D_1 P = C_2$ and $A_2 C_2 D_2 P = C_3$. If yes, compute all instances of such a common permutation matrix P.

Fact 2. *DmLCE (Problem 4)* $\leq_p$ *2-DmLCE (Problem 6)* $\leq_p$ *MCE (Problem 1).*

Proof. The reduction from Problem 6 to Problem 1 is trivial, because we can employ the algorithm for Problem 1 as a subroutine to solve the equations in Problem 6 individually.

To see that Problem 4 reduces to Problem 6, we first enumerate the $n-1$ equations in order:

$$\begin{aligned} A_1 C_1 D_1 P &= C_2 \\ A_2 C_2 D_2 P &= C_3 \\ &\vdots \\ A_{n-1} C_{n-1} D_{n-1} P &= C_n \end{aligned} \tag{16}$$

Equivalently, this can be rewritten to another system of equations:

$$\begin{aligned} A'_1 C_1 D'_1 P &= C_2 \\ A'_2 C_1 D'_2 P^2 &= C_3 \\ &\vdots \\ A'_{n-1} C_1 D'_{n-1} P^{n-1} &= C_n \end{aligned}$$

Now we pair the ith equation and the $(n-i)$th equation for each $i \in \lfloor \frac{n}{2} \rfloor$, i.e.,

$$\begin{aligned} A'_i C_1 D'_i P^i &= C_{i+1} \\ A'_{n-i} C_1 D'_{n-i} P^{n-i} &= C_{n-i+1} \end{aligned} \tag{17}$$

Assuming the order of P is n (by a random polynomial reduction; see Remark 6), we can rewrite (17) as

$$\begin{aligned} A'_i C_1 D'_i P^i &= C_{i+1} \\ A''_{n-i} C_{n-i+1} D''_{n-i} P^i &= C_1 \end{aligned} \tag{18}$$

Thus, (18) precisely recovers Problem 6, so we can employ the algorithm for Problem 6 as a subroutine to solve the equations in (16) pair-by-pair. If n is odd, then we are done. If n is even, we need to deal with one more case because $A'_{\frac{n}{2}} C_1 D'_{\frac{n}{2}} P^{\frac{n}{2}} = C_{\frac{n}{2}+1}$ is not paired. To this end, we may treat this single equation as the following system of equations that still follows the pattern of Problem 4:

$$\begin{aligned} A'_{\frac{n}{2}} C_1 D'_{\frac{n}{2}} P^{\frac{n}{2}} &= C_{\frac{n}{2}+1} \\ A''_{\frac{n}{2}} C_{\frac{n}{2}+1} D''_{\frac{n}{2}} P^{\frac{n}{2}} &= C_1. \end{aligned}$$

This concludes the proof. □

References

[AEK+22] Michel Abdalla, Thorsten Eisenhofer, Eike Kiltz, Sabrina Kunzweiler, and Doreen Riepel. Password-authenticated key exchange from group actions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 699–728. Springer, 2022.

[AFMP20] Navid Alamati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis. Cryptographic group actions and applications. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 411–439. Springer, 2020.

[Bab16] László Babai. Graph isomorphism in quasipolynomial time [extended abstract]. In *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2016, Cambridge, MA, USA, June 18-21, 2016*, pages 684–697, 2016.

[Bab19] László Babai. Canonical form for graphs in quasipolynomial time: preliminary report. In Moses Charikar and Edith Cohen, editors, *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Phoenix, AZ, USA, June 23-26, 2019*, pages 1237–1246. ACM, 2019.

[BBB+23] Marco Baldi, Alessandro Barenghi, Luke Beckwith, Jean-François Biasse, Andre Esser, Kris Gaj, Kamyar Mohajerani, Gerardo Pelosi, Edoardo Persichetti, Markku-Juhani Saarinen, Paolo Santini, and Robert Wallace. LESS: Linear equivalence signature scheme, 2023.

[BBB+25] Huck Bennett, Drisana Bhatia, Jean-François Biasse, Medha Durisheti, Lucas LaBuff, Vincenzo Pallozzi Lavorante, and Philip Waitkevich. Asymptotic improvements to provable algorithms for the code equivalence problem. *Cryptology ePrint Archive*, 2025.

[BBCDK24] Benjamin Benčina, Alessandro Budroni, Jesús-Javier Chi-Domínguez, and Mukul Kulkarni. Properties of lattice isomorphism as a cryptographic group action. In *International Conference on Post-Quantum Cryptography*, pages 170–201. Springer, 2024.

[BBMP24] Michele Battagliola, Giacomo Borin, Alessio Meneghetti, and Edoardo Persichetti. Cutting the grass: Threshold group action signature schemes. In *Cryptographers' Track at the RSA Conference*, pages 460–489. Springer, 2024.

[BCD+24a] Markus Bläser, Zhili Chen, Dung Hoang Duong, Antoine Joux, Tuong Ngoc Nguyen, Thomas Plantard, Youming Qiao, Willy Susilo, and Gang Tang. On digital signatures based on group actions: QROM security and ring signatures. In Markku-Juhani O. Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Oxford, UK, June 12-14, 2024, Proceedings, Part I*, volume 14771 of *Lecture Notes in Computer Science*, pages 227–261. Springer, 2024.

[BCD+24b] Alessandro Budroni, Jesús-Javier Chi-Domínguez, Giuseppe D'Alconzo, Antonio J. Di Scala, and Mukul Kulkarni. Don't use it twice! solving relaxed linear equivalence problems. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VIII*, volume 15491 of *Lecture Notes in Computer Science*, pages 35–65. Springer, 2024.

[BCGQ11] László Babai, Paolo Codenotti, Joshua A. Grochow, and Youming Qiao. Code equivalence and group isomorphism. In *Proceedings of the Twenty-Second Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2011, San Francisco, California, USA, January 23-25, 2011*, pages 1395–1408, 2011.

[BCK98] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. A modular approach to the design and analysis of authentication and key exchange protocols (extended abstract). In Jeffrey Scott Vitter, editor, *Proceedings of the Thirtieth Annual ACM Symposium on the Theory of Computing, Dallas, Texas, USA, May 23-26, 1998*, pages 419–428. ACM, 1998.

[BCP97] Wieb Bosma, John Cannon, and Catherine Playoust. The Magma algebra system. I. The user language. *J. Symbolic Comput.*, 24(3-4):235–265, 1997. Computational algebra and number theory (London, 1993).

[BDFGP23] Ward Beullens, Luca De Feo, Steven D Galbraith, and Christophe Petit. Proving knowledge of isogenies: a survey. *Designs, Codes and Cryptography*, 91(11):3425–3456, 2023.

[BDK+22] Ward Beullens, Samuel Dobson, Shuichi Katsumata, Yi-Fu Lai, and Federico Pintore. Group signatures and more from isogenies and lattices: Generic, simple, and efficient.

In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 95–126. Springer, 2022.

[BDN+23] Markus Bläser, Dung Hoang Duong, Anand Kumar Narayanan, Thomas Plantard, Youming Qiao, Arnaud Sipasseuth, and Gang Tang. The ALTEQ Signature Scheme: Algorithm Specifications and Supporting Documentation. NIST Post-Quantum Additional Digital Signature Schemes Submission, 2023. Authors listed on the official NIST specification document.

[BEFN25] Alessandro Budroni, Andre Esser, Ermes Franch, and Andrea Natale. Two is all it takes: Asymptotic and concrete improvements for solving code equivalence. *Cryptology ePrint Archive*, 2025.

[Beu20] Ward Beullens. Not enough LESS: an improved algorithm for solving code equivalence problems over $\mathbb{F}_q$. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O'Flynn, editors, *Selected Areas in Cryptography - SAC 2020 - 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers*, volume 12804 of *Lecture Notes in Computer Science*, pages 387–403. Springer, 2020.

[BFFP11] Charles Bouillaguet, Jean-Charles Faugère, Pierre-Alain Fouque, and Ludovic Perret. Practical cryptanalysis of the identification scheme based on the isomorphism of polynomial with one secret problem. In *International Workshop on Public Key Cryptography*, pages 473–493. Springer, 2011.

[BFV13] Charles Bouillaguet, Pierre-Alain Fouque, and Amandine Véber. Graph-theoretic algorithms for the "isomorphism of polynomials" problem. In *Advances in Cryptology - EUROCRYPT 2013*, pages 211–227, 2013.

[BGZ23] Dan Boneh, Jiaxin Guan, and Mark Zhandry. A lower bound on the length of signatures based on group actions and generic isogenies. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 507–531. Springer, 2023.

[BKP20] Ward Beullens, Shuichi Katsumata, and Federico Pintore. Calamari and falafl: Logarithmic (linkable) ring signatures from isogenies and lattices. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 464–492. Springer, 2020.

[BKV19] Ward Beullens, Thorsten Kleinjung, and Frederik Vercauteren. CSI-FiSh: Efficient isogeny based signatures through class group computations. In *Advances in Cryptology - ASIACRYPT 2019*, volume 11921 of *Lecture Notes in Computer Science*, pages 227–247. Springer, 2019.

[BMPS20] Jean-François Biasse, Giacomo Micheli, Edoardo Persichetti, and Paolo Santini. LESS is more: Code-based signatures without syndromes. In Abderrahmane Nitaj and

Amr M. Youssef, editors, *Progress in Cryptology - AFRICACRYPT 2020 - 12th International Conference on Cryptology in Africa, Cairo, Egypt, July 20-22, 2020, Proceedings*, volume 12174 of *Lecture Notes in Computer Science*, pages 45–65. Springer, 2020.

[BN25] Alessandro Budroni and Andrea Natale. On the sample complexity of linear code equivalence for all code rates. *Cryptography and Communications*, pages 1–23, 2025.

[BNZ25] John Bostanci, Barak Nehoran, and Mark Zhandry. A general quantum duality for representations of groups with applications to quantum money, lightning, and fire. In *Proceedings of the 57th ACM Symposium on Theory of Computing (STOC 2025)*, pages 189–200, 2025.

[Bon98] Dan Boneh. The decision Diffie-Hellman problem. In *Algorithmic Number Theory, Third International Symposium, ANTS-III, Portland, Oregon, USA, June 21-25, 1998, Proceedings*, pages 48–63, 1998.

[BOS19] Magali Bardet, Ayoub Otmani, and Mohamed Saeed-Taha. Permutation code equivalence is not harder than graph isomorphism when hulls are trivial. In *IEEE International Symposium on Information Theory, ISIT 2019, Paris, France, July 7-12, 2019*, pages 2464–2468. IEEE, 2019.

[Bou11] Charles Bouillaguet. *Etudes d'hypotheses algorithmiques et attaques de primitives cryptographiques*. PhD thesis, PhD thesis, PhD thesis, Université Paris-Diderot–École Normale Supérieure, 2011.

[BR93] Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *Advances in Cryptology - CRYPTO '93, 13th Annual International Cryptology Conference, Santa Barbara, California, USA, August 22-26, 1993, Proceedings*, volume 773 of *Lecture Notes in Computer Science*, pages 232–249. Springer, 1993.

[BS92] László Babai and Ákos Seress. On the diameter of permutation groups. *European journal of combinatorics*, 13(4):231–243, 1992.

[BS20] Xavier Bonnetain and André Schrottenloher. Quantum security analysis of CSIDH. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part II*, volume 12106 of *Lecture Notes in Computer Science*, pages 493–522. Springer, 2020.

[BT19] Henry Bradford and Andreas Thom. Short laws for finite groups and residual finiteness growth. *Transactions of the American Mathematical Society*, 371(9):6447–6462, 2019.

[BY90] Gilles Brassard and Moti Yung. One-way group actions. In *Advances in Cryptology - CRYPTO 1990*, pages 94–107, 1990.

[Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA*, pages 136–145. IEEE Computer Society, 2001.

[CJS14] Andrew Childs, David Jao, and Vladimir Soukharev. Constructing elliptic curve isogenies in quantum subexponential time. *Journal of Mathematical Cryptology*, 8(1):1–29, 2014.

[CLM+18] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. CSIDH: an efficient post-quantum commutative group action. In *International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, pages 395–427. Springer, 2018.

[CNP+23] Tung Chou, Ruben Niederhagen, Edoardo Persichetti, Tovohery Hajatiana Randrianarisoa, Krijn Reijnders, Simona Samardjiska, and Monika Trimoska. Take your meds: Digital signatures from matrix code equivalence. In *Progress in Cryptology - AFRICACRYPT 2023*, 2023.

[Cou06] Jean Marc Couveignes. Hard homogeneous spaces. *IACR Cryptology ePrint Archive*, 2006.

[CPS25] Tung Chou, Edoardo Persichetti, and Paolo Santini. On linear equivalence, canonical forms, and digital signatures. *Designs, Codes and Cryptography*, pages 1–43, 2025.

[CvD10] Andrew M. Childs and Wim van Dam. Quantum algorithms for algebraic problems. *Rev. Mod. Phys.*, 82:1–52, Jan 2010.

[DDMW06] Anupam Datta, Ante Derek, John C. Mitchell, and Bogdan Warinschi. Key exchange protocols: Security definition, proof method and applications. Cryptology ePrint Archive, Paper 2006/056, 2006.

[DDS24] Giuseppe D'Alconzo and Antonio J Di Scala. Representations of group actions and their applications in cryptography. *Finite Fields and Their Applications*, 99:102476, 2024.

[DH76] Whitfield Diffie and Martin Hellman. New directions in cryptography. *IEEE transactions on Information Theory*, 22(6):644–654, 1976.

[DKQ+25] Dung Hoang Duong, Xuan Thanh Khuc, Youming Qiao, Willy Susilo, and Chuanqi Zhang. Blind signatures from cryptographic group actions. *Cryptology ePrint Archive*, 2025.

[DMR11] Hang Dinh, Cristopher Moore, and Alexander Russell. McEliece and Niederreiter cryptosystems that resist quantum Fourier sampling attacks. In *Advances in Cryptology – CRYPTO 2011*, pages 761–779, 2011.

[DMR15] Hang T. Dinh, Cristopher Moore, and Alexander Russell. Limitations of single coset states and quantum algorithms for code equivalence. *Quantum Information & Computation*, 15(3&4):260–294, 2015.

[DMS24] Giuseppe D'Alconzo, Alessio Meneghetti, and Edoardo Signorini. Group factorisation for smaller signatures from cryptographic group actions. *Cryptology ePrint Archive*, 2024.

[DPPvW22] Léo Ducas, Eamonn W. Postlethwaite, Ludo N. Pulles, and Wessel van Woerden. HAWK: Module lip makes lattice signatures fast, compact and simple. In *Advances in*

Cryptology – ASIACRYPT 2022, volume 13794 of *Lecture Notes in Computer Science*, pages 65–94. Springer, 2022.

[dSGFW20] Cyprien Delpech de Saint Guilhem, Marc Fischlin, and Bogdan Warinschi. Authentication in key-exchange: Definitions, relations and composition. In *33rd IEEE Computer Security Foundations Symposium, CSF 2020, Boston, MA, USA, June 22-26, 2020*, pages 288–303. IEEE, 2020.

[ElG85] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE transactions on information theory*, 31(4):469–472, 1985.

[FG19] Luca De Feo and Steven D. Galbraith. Seasign: Compact isogeny signatures from class group actions. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019*, volume 11478 of *Lecture Notes in Computer Science*, pages 759–789. Springer, 2019.

[FGSW16] Marc Fischlin, Felix Günther, Benedikt Schmidt, and Bogdan Warinschi. Key confirmation in key exchange: A formal treatment and implications for TLS 1.3. In *IEEE Symposium on Security and Privacy, SP 2016, San Jose, CA, USA, May 22-26, 2016*, pages 452–469. IEEE Computer Society, 2016.

[FM20] Luca De Feo and Michael Meyer. Threshold schemes from isogeny assumptions. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *Public-Key Cryptography - PKC 2020 - 23rd IACR International Conference on Practice and Theory of Public-Key Cryptography, Edinburgh, UK, May 4-7, 2020, Proceedings, Part II*, volume 12111 of *Lecture Notes in Computer Science*, pages 187–212. Springer, 2020.

[FS86] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – CRYPTO 1986*, pages 186–194, 1986.

[GMW91] Oded Goldreich, Silvio Micali, and Avi Wigderson. Proofs that yield nothing but their validity for all languages in NP have zero-knowledge proof systems. *J. ACM*, 38(3):691–729, 1991.

[GQ19] Joshua A. Grochow and Youming Qiao. Isomorphism problems for tensors, groups, and cubic forms: completeness and reductions, 2019. arXiv:1907.00309 [cs.CC].

[HMR+10] Sean Hallgren, Cristopher Moore, Martin Rötteler, Alexander Russell, and Pranab Sen. Limitations of quantum coset states for graph isomorphism. *J. ACM*, 57(6):34:1–34:33, November 2010.

[HMY23] Minki Hhan, Tomoyuki Morimae, and Takashi Yamakawa. From the hardness of detecting superpositions to cryptography: Quantum public key encryption and commitments. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 639–667. Springer, 2023.

[Hou25] Marc Houben. Deterministic algorithms for class group actions. In *Advances in Cryptology – CRYPTO 2025*, volume 16000 of *Lecture Notes in Computer Science*, pages 100–130. Springer, 2025.

[HR14] Ishay Haviv and Oded Regev. On the lattice isomorphism problem. In *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2014, Portland, Oregon, USA, January 5-7, 2014*, pages 391–404, 2014.

[HS14] Harald A Helfgott and Ákos Seress. On the diameter of permutation groups. *Annals of mathematics*, pages 611–658, 2014.

[IY23] Ren Ishibashi and Kazuki Yoneyama. Compact password authenticated key exchange from group actions. In Leonie Simpson and Mir Ali Rezazadeh Baee, editors, *Information Security and Privacy - 28th Australasian Conference, ACISP 2023, Brisbane, QLD, Australia, July 5-7, 2023, Proceedings*, volume 13915 of *Lecture Notes in Computer Science*, pages 220–247. Springer, 2023.

[JQSY19] Zhengfeng Ji, Youming Qiao, Fang Song, and Aaram Yun. General linear group action on tensors: A candidate for post-quantum cryptography. In *Theory of cryptography conference*, pages 251–281. Springer, 2019.

[KLLQ23] Shuichi Katsumata, Yi-Fu Lai, Jason T. LeGrow, and Ling Qin. CSI -otter: Isogeny-based (partially) blind signatures from the class group action with a twist. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 729–761. Springer, 2023.

[KS99] Aviad Kipnis and Adi Shamir. Cryptanalysis of the hfe public key cryptosystem by relinearization. In *Annual International Cryptology Conference*, pages 19–30. Springer, 1999.

[KT16] Gady Kozma and Andreas Thom. Divisibility and laws in finite simple groups. *Mathematische Annalen*, 364(1):79–95, 2016.

[Kup05] Greg Kuperberg. A subexponential-time quantum algorithm for the dihedral hidden subgroup problem. *SIAM J. Comput.*, 35(1):170–188, 2005.

[Leo82] Jeffrey S. Leon. Computing automorphism groups of error-correcting codes. *IEEE Trans. Information Theory*, 28(3):496–510, 1982.

[Lie84] Martin W Liebeck. On minimal degrees and base sizes of primitive permutation groups. *Archiv der Mathematik*, 43(1):11–15, 1984.

[LR24] Antonin Leroux and Maxime Roméas. Updatable encryption from group actions. In Markku-Juhani O. Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Oxford, UK, June 12-14, 2024, Proceedings, Part II*, volume 14772 of *Lecture Notes in Computer Science*, pages 20–53. Springer, 2024.

[Luk82] Eugene M. Luks. Isomorphism of graphs of bounded valence can be tested in polynomial time. *J. Comput. Syst. Sci.*, 25(1):42–65, 1982.

[MP14] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *J. Symb. Comput.*, 60:94–112, 2014.

[MRS10] Cristopher Moore, Alexander Russell, and Piotr Sniady. On the impossibility of a quantum sieve algorithm for graph isomorphism. *SIAM J. Comput.*, 39(6):2377–2396, 2010.

[MRV07] Cristopher Moore, Alexander Russell, and Umesh Vazirani. A classical one-way function to confound quantum adversaries. *arXiv preprint quant-ph/0701115*, 2007.

[MSU11] Alexei G Myasnikov, Vladimir Shpilrain, and Alexander Ushakov. *Non-commutative cryptography and complexity of group-theoretic problems*, volume 177 of *Mathematical Surveys and Monographs*. American Mathematical Soc., 2011.

[MZ22] Hart Montgomery and Mark Zhandry. Full quantum equivalence of group action dlog and cdh, and more. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 3–32. Springer, 2022.

[NP07] Alice C Niemeyer and Cheryl E Praeger. On permutations of order dividing a given integer. *Journal of Algebraic Combinatorics*, 26:125–142, 2007.

[Pei20] Chris Peikert. He gives c-sieves on the CSIDH. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part II*, volume 12106 of *Lecture Notes in Computer Science*, pages 463–492. Springer, 2020.

[PN17] Jacques Patarin and Valérie Nachef. Commutativity, associativity, and public key cryptography. In Jerzy Kaczorowski, Josef Pieprzyk, and Jacek Pomykala, editors, *Number-Theoretic Methods in Cryptology - First International Conference, NuTMiC 2017, Warsaw, Poland, September 11-13, 2017, Revised Selected Papers*, volume 10737 of *Lecture Notes in Computer Science*, pages 104–117. Springer, 2017.

[PS23] Edoardo Persichetti and Paolo Santini. A new formulation of the linear equivalence problem and shorter less signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 351–378. Springer, 2023.

[PW01] Birgit Pfitzmann and Michael Waidner. A model for asynchronous reactive systems and its application to secure message transmission. In *2001 IEEE Symposium on Security and Privacy, Oakland, California, USA May 14-16, 2001*, pages 184–200. IEEE Computer Society, 2001.

[QS24] Youming Qiao and Xiaorui Sun. Canonical forms for matrix tuples in polynomial time. In *2024 IEEE 65th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 780–789. IEEE, 2024.

[RS06] Alexander Rostovtsev and Anton Stolbunov. Public-key cryptosystem based on isogenies. *IACR Cryptol. ePrint Arch.*, page 145, 2006.

[Sen00] Nicolas Sendrier. Finding the permutation between equivalent linear codes: The support splitting algorithm. *IEEE Trans. Information Theory*, 46(4):1193–1203, 2000.

[Sho94] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*, pages 124–134, 1994.

[Sho99] Victor Shoup. On formal models for secure key exchange. *IACR Cryptol. ePrint Arch.*, page 12, 1999.

[Sto10] Anton Stolbunov. Constructing public-key cryptographic schemes based on class group action on a set of isogenous elliptic curves. *Adv. Math. Commun.*, 4(2):215–235, 2010.

[TDJ+22] Gang Tang, Dung Hoang Duong, Antoine Joux, Thomas Plantard, Youming Qiao, and Willy Susilo. Practical post-quantum signature schemes from isomorphism problems of trilinear forms. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 582–612. Springer, 2022.

[Tho17] Andreas Thom. About the length of laws for finite groups. *Israel Journal of Mathematics*, 219:469–478, 2017.

[Zyr20] Christiane Zyrus. *Almost laws for finite simple groups.* PhD thesis, Technische Universität Dresden, 2020.